Data Structures (in C++)

- Sorting -

부산대학교 정보·의생명공학대학
정보컴퓨터공학부

Sorting

Sorting

- **Sorting**
  - The sorting problem is to reorder the sequence so that the elements are in (non-)decreasing order
  - Rearrange items to be ordered by some criterion (*e.g.*, ascending order)

- **Sorting Algorithms**
  - Insertion-Sort
  - Selection-Sort

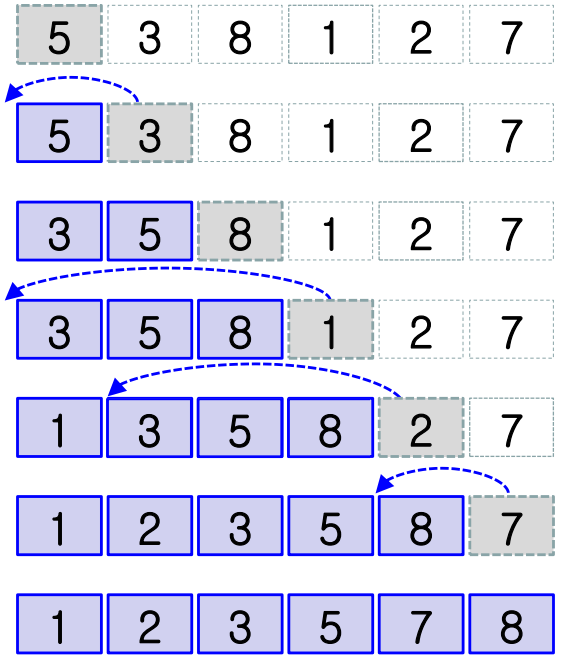
  } Already covered in previous lectures

  - Bubble-Sort
  - Heap-Sort
  - Merge-Sort
  - Quick-Sort
  - Bucket-Sort
  - Radix-Sort

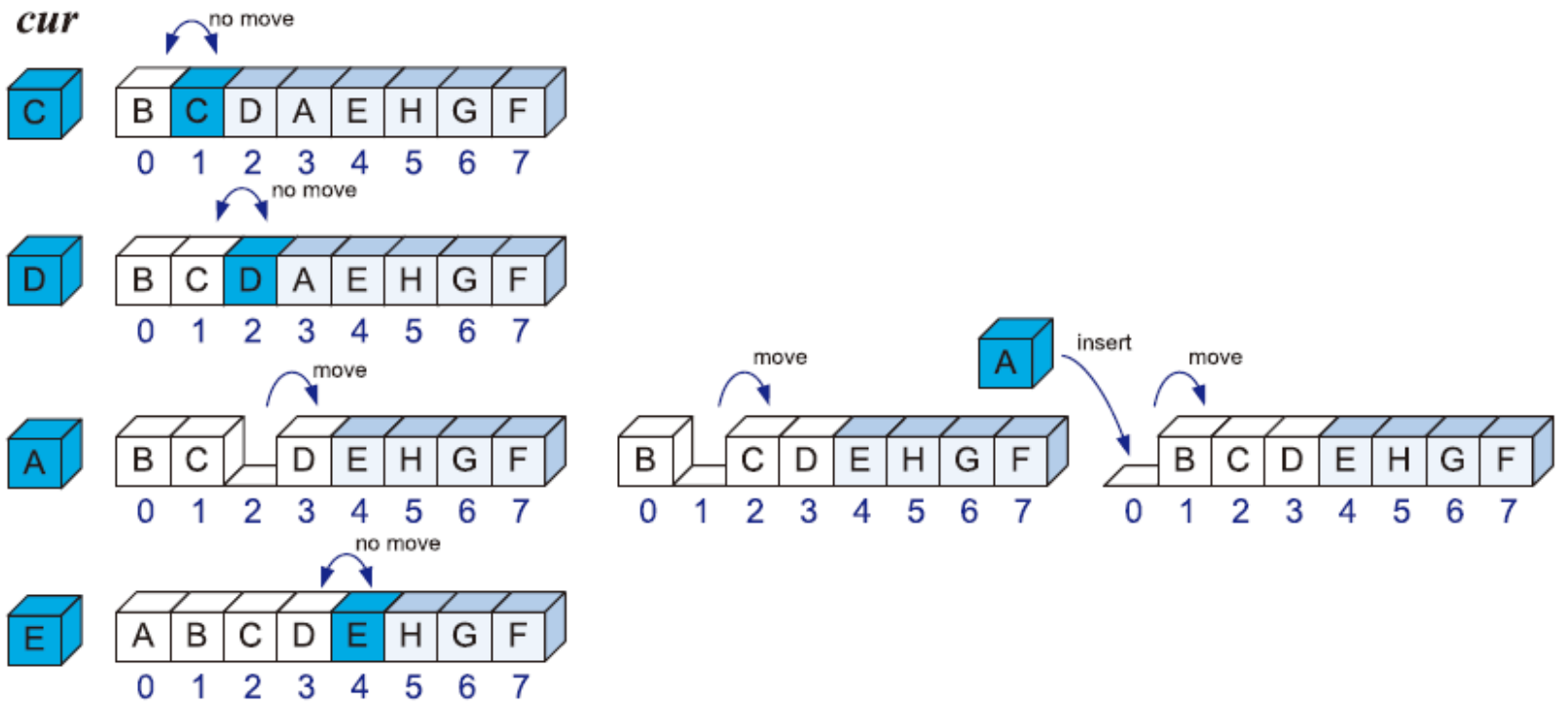
- **Insertion-Sort**
  - Each iteration of the algorithm inserts the next element into the current sorted part of the array

Algorithm InsertionSort(A):
Input: An array A of n comparable elements
Output: The array A with elements rearranged in nondecreasing order
for $i \leftarrow 1$ to $n-1$ **do**
{Insert $A[i]$ at its proper location in $A[0], A[1], \ldots, A[i-1]$}
$cur \leftarrow A[i]$
$j \leftarrow i-1$
while $j \geq 0$ and $A[j] > cur$ **do**
$A[j+1] \leftarrow A[j]$
$j \leftarrow j-1$
$A[j+1] \leftarrow cur$ {cur is now in the right place}

Pseudocode

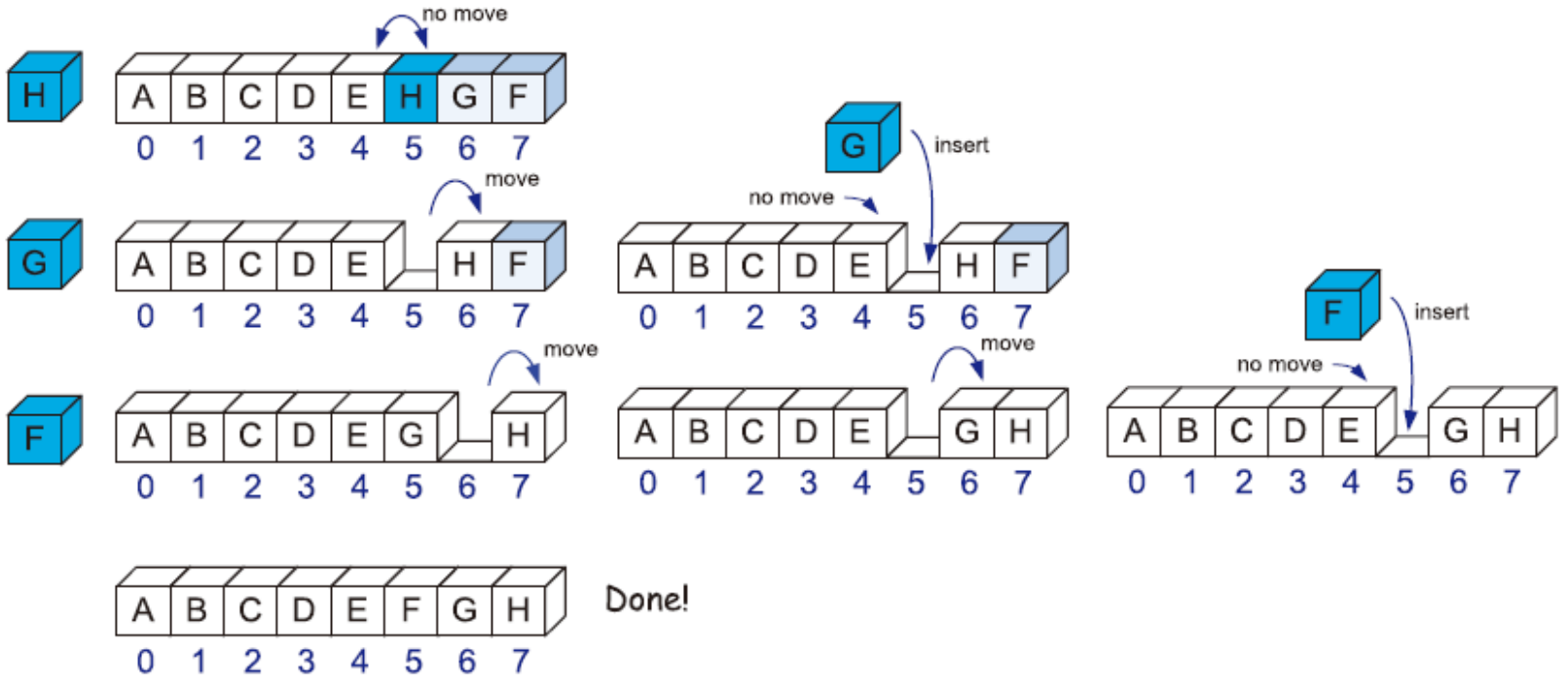


Insertion-Sort



https://www.cs.usfca.edu/~galles/visualization/ComparisonSort.html

Insertion-Sort



Selection-Sort

- **Selection-Sort**
  - Each iteration of the algorithm selects the next element to be inserted into the current sorted array

		List L	*Priority Queue P*
Input		$(7,4,8,2,5,3,9)$	$()$
Phase 1	(a)	$(4,8,2,5,3,9)$	(7)
$O(n)$	(b)	$(8,2,5,3,9)$	$(7,4)$
	⋮	⋮	⋮
	(g)	$()$	$(7,4,8,2,5,3,9)$
Phase 2	(a)	(2)	$(7,4,8,5,3,9)$
$O(n^2)$	(b)	$(2,3)$	$(7,4,8,5,9)$
	(c)	$(2,3,4)$	$(7,8,5,9)$
	(d)	$(2,3,4,5)$	$(7,8,9)$
	(e)	$(2,3,4,5,7)$	$(8,9)$
	(f)	$(2,3,4,5,7,8)$	(9)
	(g)	$(2,3,4,5,7,8,9)$	$()$

Algorithm PriorityQueueSort(L,P):
Input: An STL list L of n elements and a priority queue, P, that compares elements using a total order relation
Output: The sorted list L

```
while !L.empty() do
  e ← L.front
  L.pop_front()        {remove an element e from the list}
  P.insert(e)          {...and it to the priority queue}
while !P.empty() do
  e ← P.min()
  P.removeMin()        {remove the smallest element e from the queue}
  L.push_back(e)       {...and append it to the back of L}
```

Bubble-Sort

- **Bubble-Sort**
  - The bubble-sort performs a series of passes over the sequence
  - In each pass, the elements are scanned from the lowest rank and compared to their neighbors
  - Swap if two consecutive neighbors are found to be in the wrong relative order

pass	swaps	sequence
		$(5,7,2,6,9,3)$
1st	$7 \leftrightarrow 2$ $7 \leftrightarrow 6$ $9 \leftrightarrow 3$	$(5,2,6,7,3,9)$
2nd	$5 \leftrightarrow 2$ $7 \leftrightarrow 3$	$(2,5,6,3,7,9)$
3rd	$6 \leftrightarrow 3$	$(2,5,3,6,7,9)$
4th	$5 \leftrightarrow 3$	$(2,3,5,6,7,9)$

- **Bubble-Sort**

  - In the first pass, once the largest element is reached, it keeps on being swapped until it gets to the last position of the sequence.
  - In the second pass, once the second largest element is reached, it keeps on being swapped until it gets to the second-to-last position of the sequence.
  - In general, at the end of the ith pass, the right-most i elements of the sequence (that is, those at indices from $n-1$ down to $n-i$) are in final position.

  - The number of passes should be n for a bubble-sort on an n-element sequence
  - *i-th* pass can be limited to the first $(n-i+1)$ elements of the sequence
    - The running time of the *i-th* pass is $O(n-i+1)$
  - The overall running time of bubble-sort is:

Assume that accesses and swaps can each be implemented in $O(1)$ time

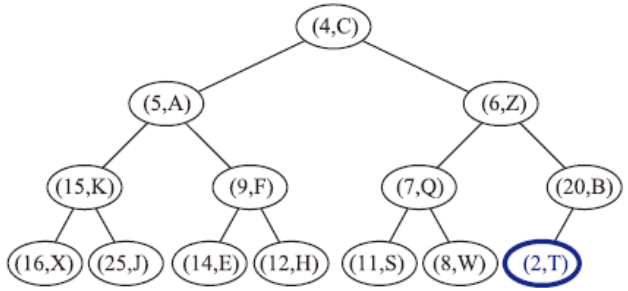
$$O\left(\sum_{i=1}^{n}(n-i+1)\right) = O(n+(n-1)+\cdots+2+1) = O\left(\sum_{i=1}^{n} i\right) = O\left(\frac{n(n+1)}{2}\right) = O(n^2)$$

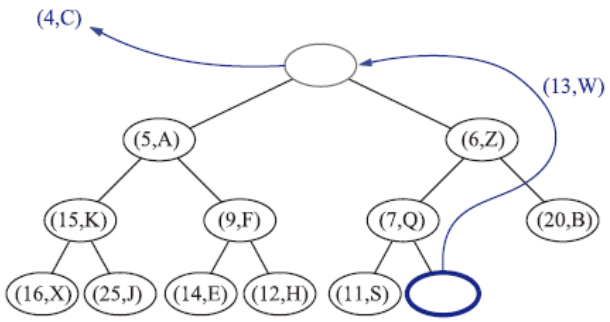
Heap-Sort

1. In the first phase, we put the elements of L into an initially empty priority queue P through a series of n insert operations, one for each element.
2. In the second phase, we extract the elements from P in nondecreasing order by means of a series of n combinations of min and removeMin operations, putting them back into L in order.

Algorithm PriorityQueueSort(L, P):
Input: An STL list L of n elements and a priority queue, P, that compares elements using a total order relation
Output: The sorted list L

```
while !L.empty() do
    e ← L.front
    L.pop_front()          {remove an element e from the list}
    P.insert(e)            {... and it to the priority queue}
while !P.empty() do
    e ← P.min()
    P.removeMin()          {remove the smallest element e from the queue}
    L.push_back(e)         {... and append it to the back of L}
```





Merge-Sort

Divide-and-Conquer

- **Divide-and-Conquer**
  - Split a problem into smaller problems and combine results

1. ***Divide:*** If the input size is smaller than a certain threshold (say, one or two elements), solve the problem directly using a straightforward method and return the solution so obtained. Otherwise, divide the input data into two or more disjoint subsets.
2. ***Conquer:*** Recursively solve the subproblems associated with the subsets.
3. ***Combine:*** Take the solutions to the subproblems and merge them into a solution to the original problem.

- **Divide-and-Conquer for Sorting**

1. ***Divide:*** If S has zero or one element, return S immediately; it is already sorted. Otherwise (S has at least two elements), remove all the elements from S and put them into two sequences, S_1 and S_2, each containing about half of the elements of S; that is, S_1 contains the first $\lfloor n/2 \rfloor$ elements of S, and S_2 contains the remaining $\lceil n/2 \rceil$ elements.
2. ***Conquer:*** Recursively sort sequences S_1 and S_2.
3. ***Combine:*** Put the elements back into S by merging the sorted sequences S_1 and S_2 into a sorted sequence.

Merge-Sort

- **Merge-Sort Tree**
  - Visualize an execution of the merge-sort algorithm using a binary tree T
  - Each node of T represents a recursive call of the merge-sort
  - The height of the merge-sort tree is $O(\log n)$

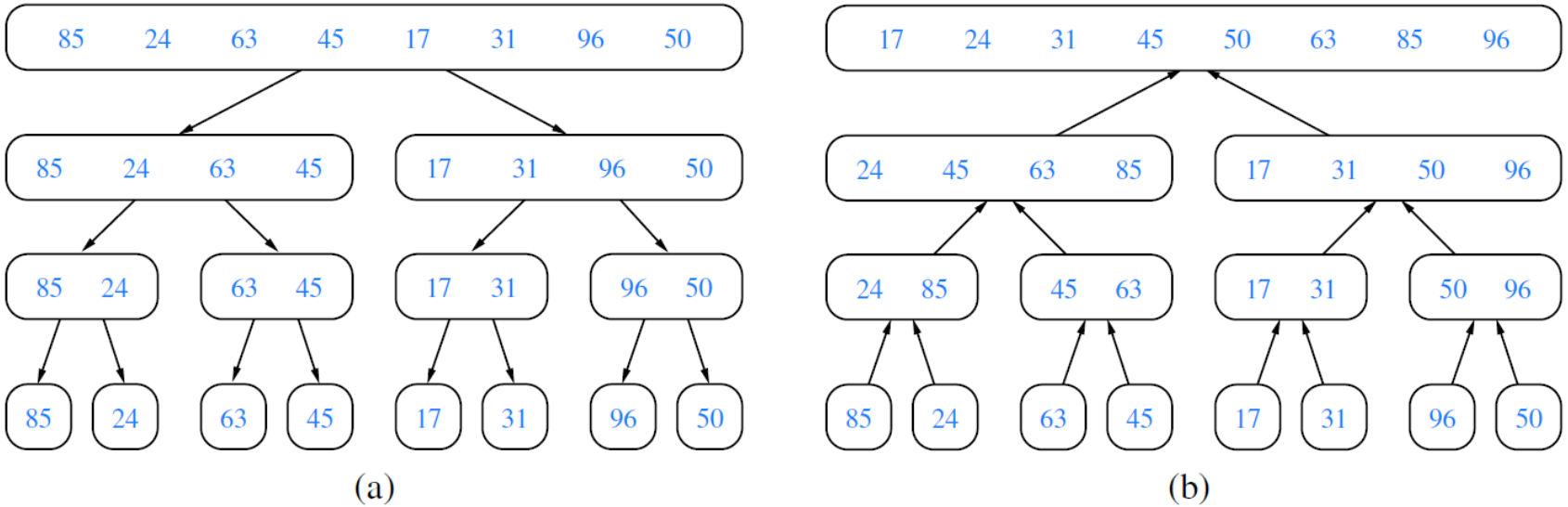
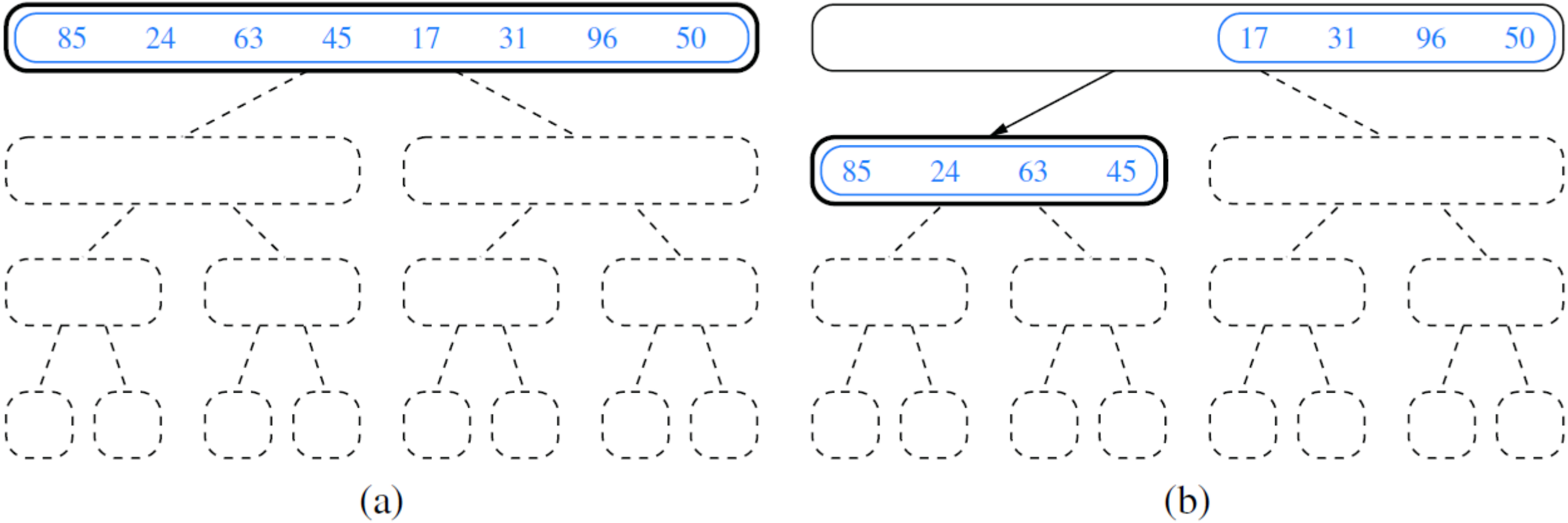


Figure 12.1: Merge-sort tree T for an execution of the merge-sort algorithm on a sequence with 8 elements: (a) input sequences processed at each node of T; (b) output sequences generated at each node of T.

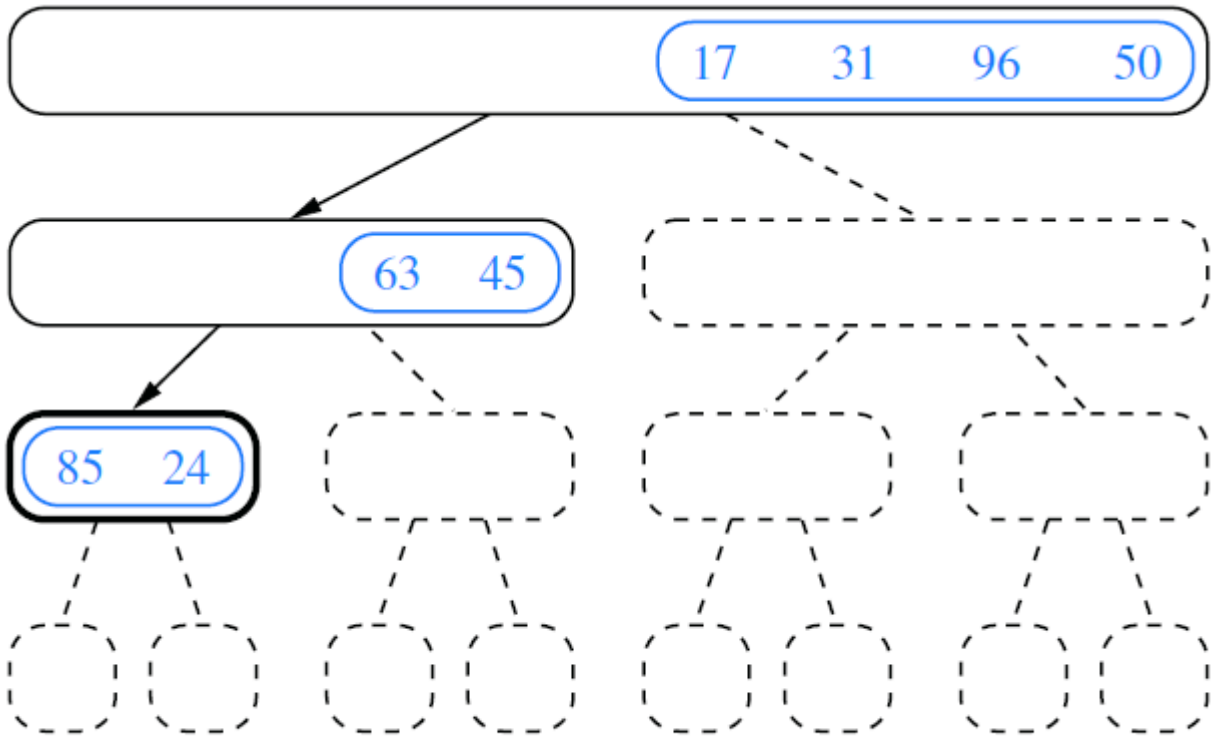
Merge-Sort

- **Merge-Sort Example**

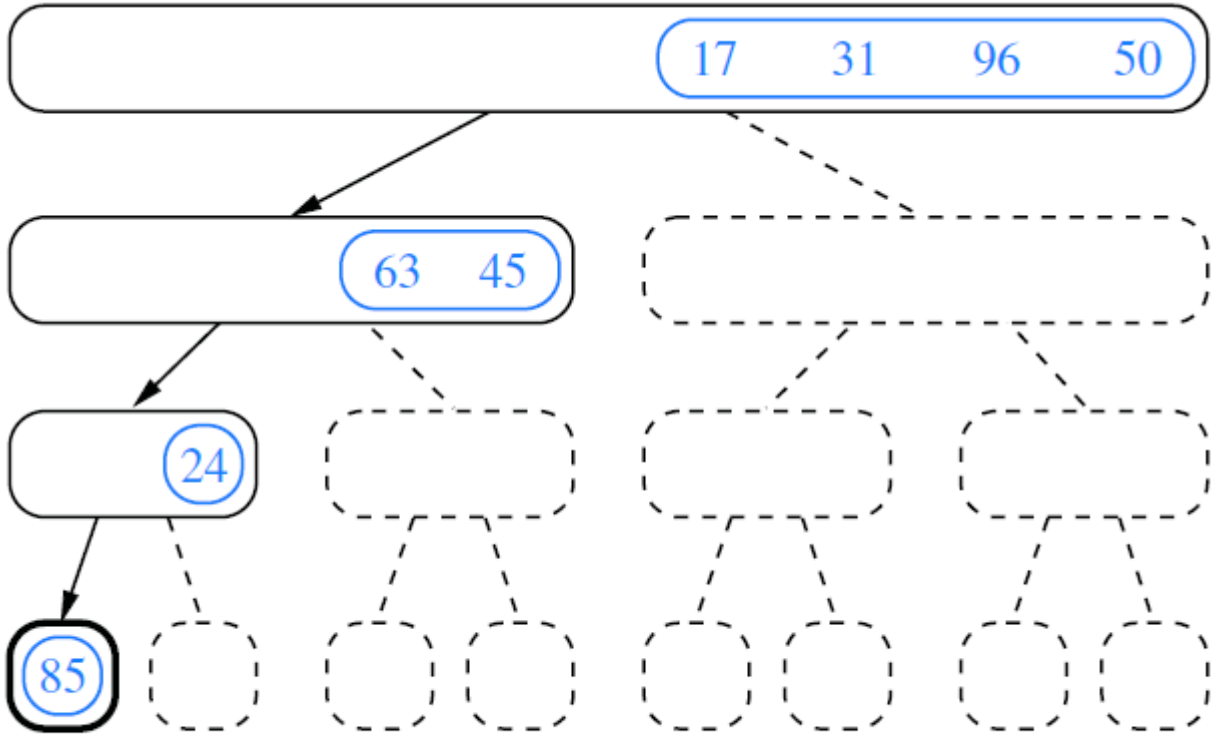


Merge-Sort

- **Merge-Sort Example**



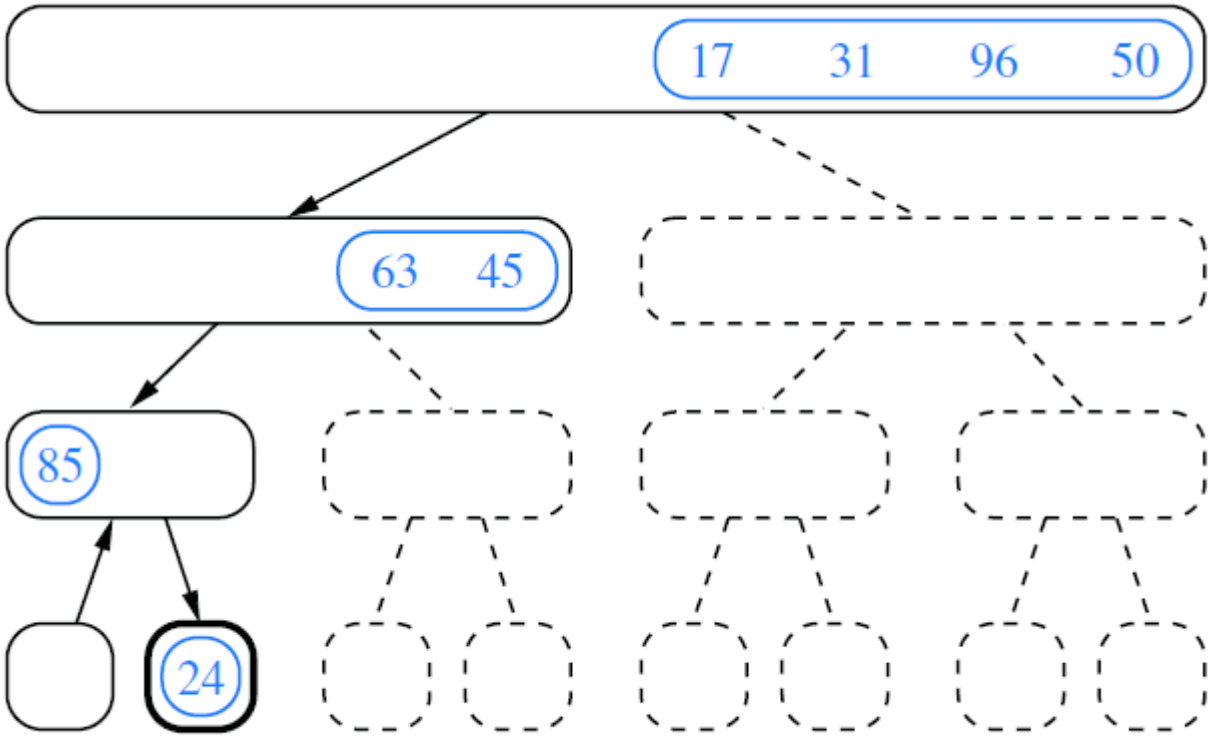
(c)



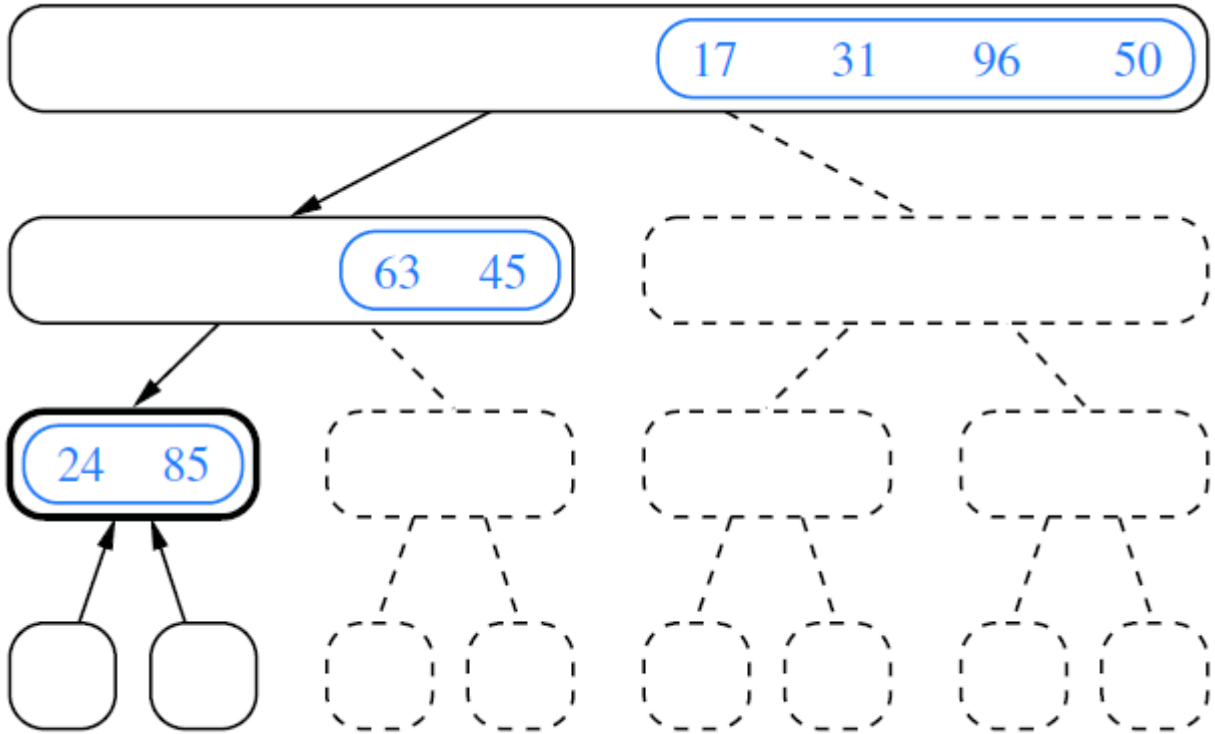
(d)

Merge-Sort

- **Merge-Sort Example**



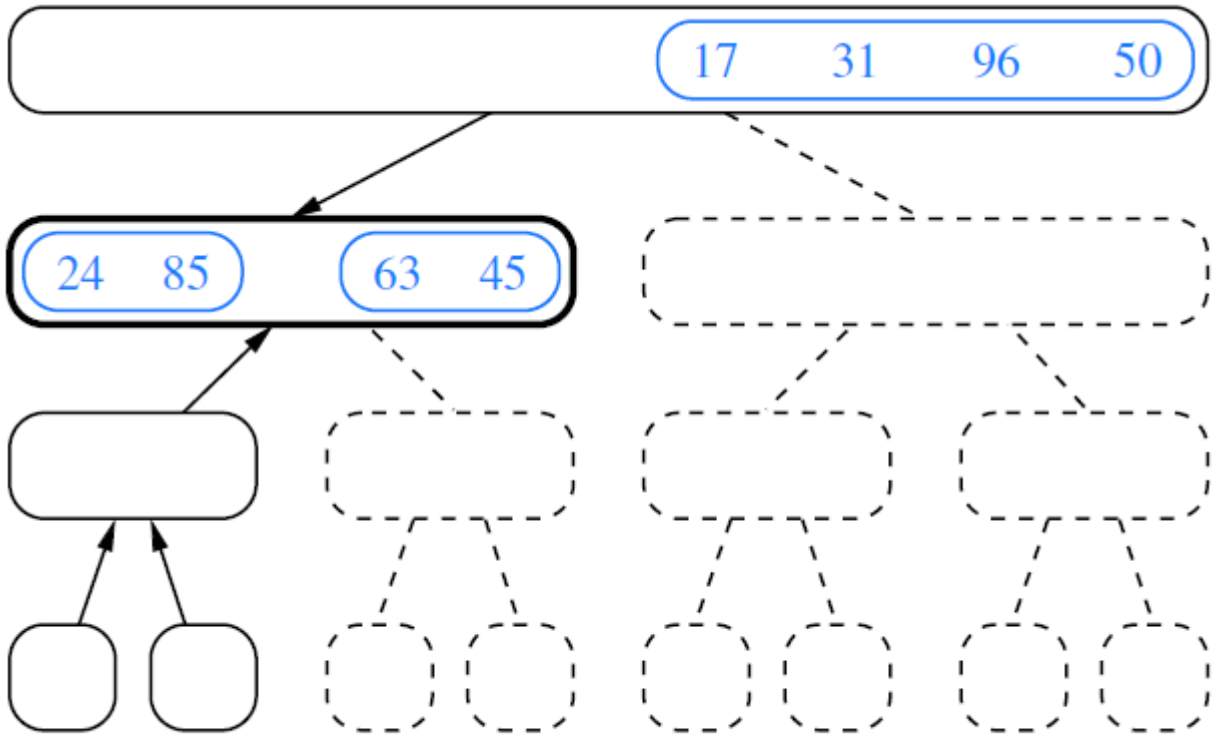
(e)



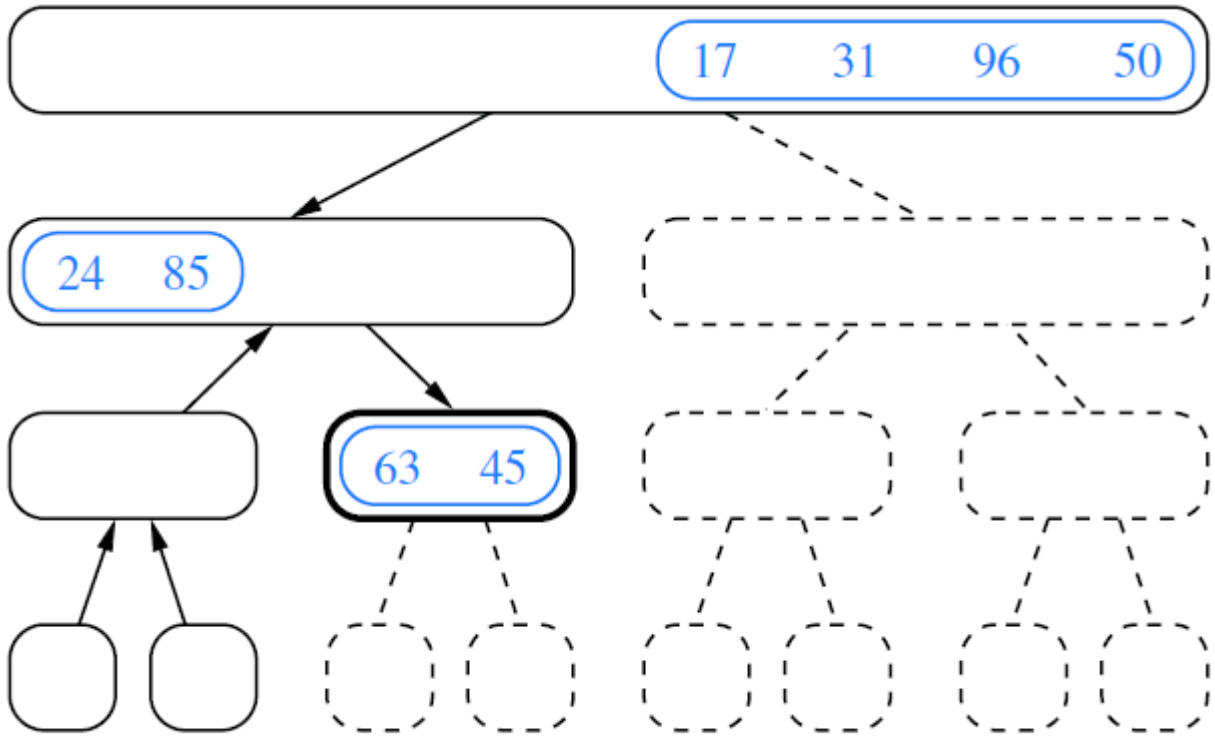
(f)

Merge-Sort

- **Merge-Sort Example**



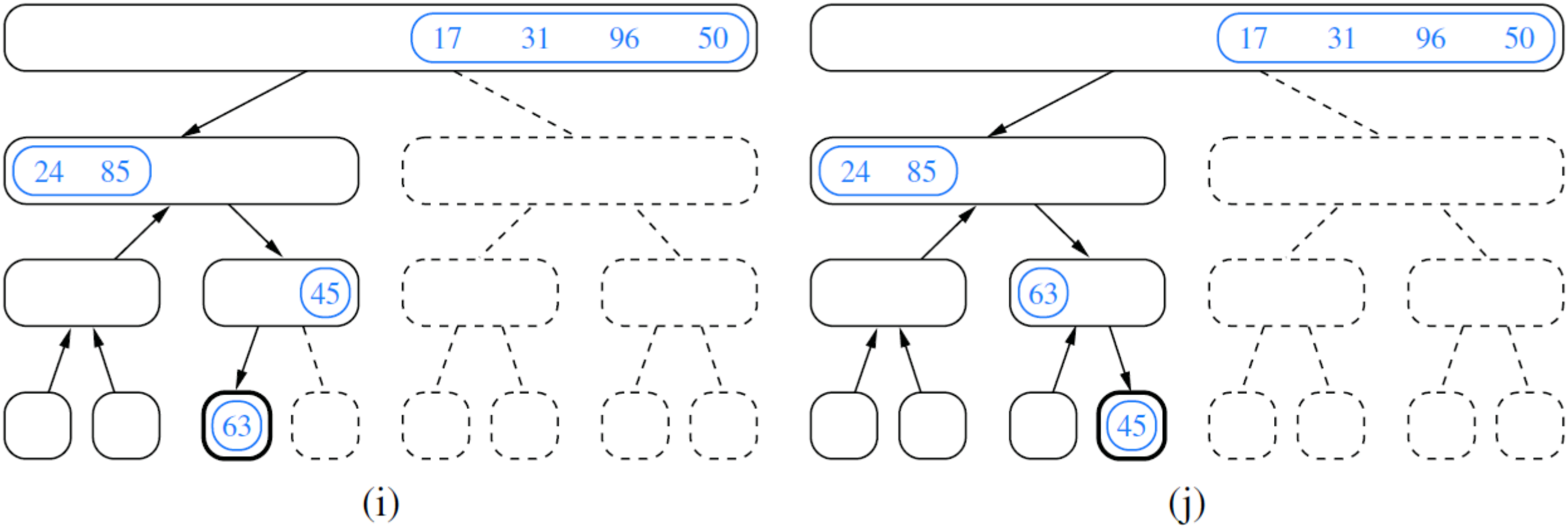
(g)



(h)

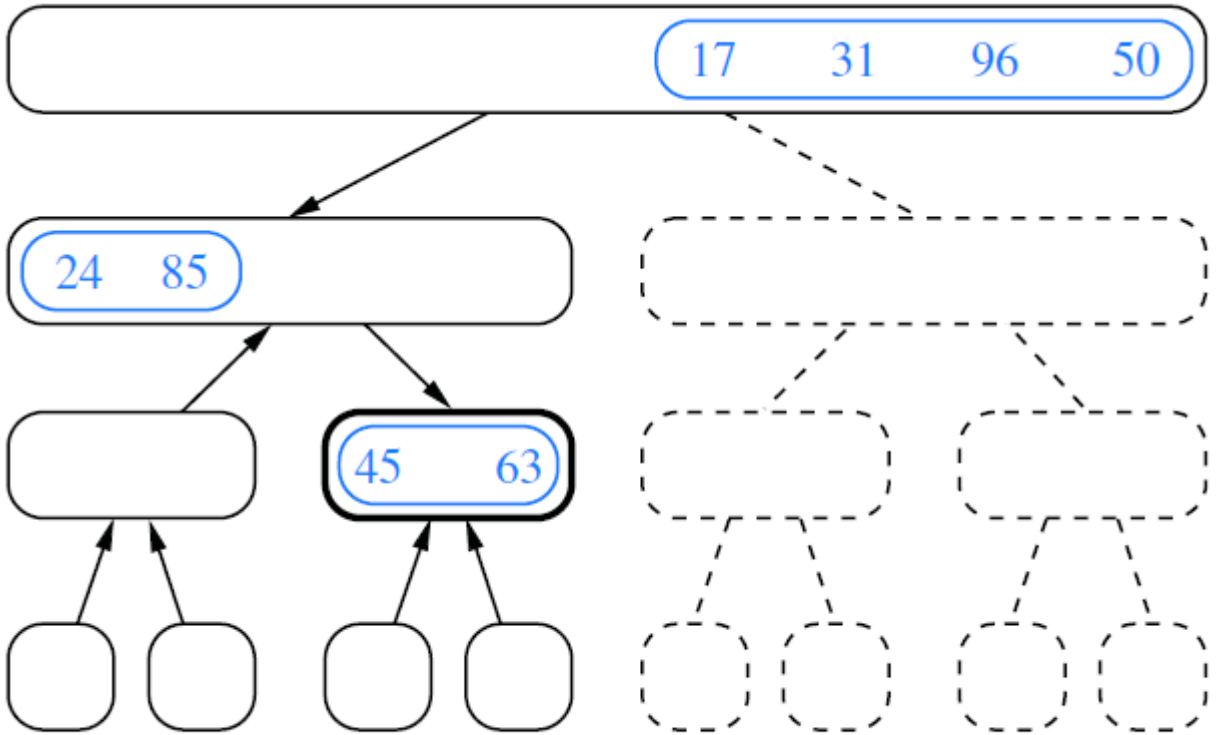
Merge-Sort

- **Merge-Sort Example**

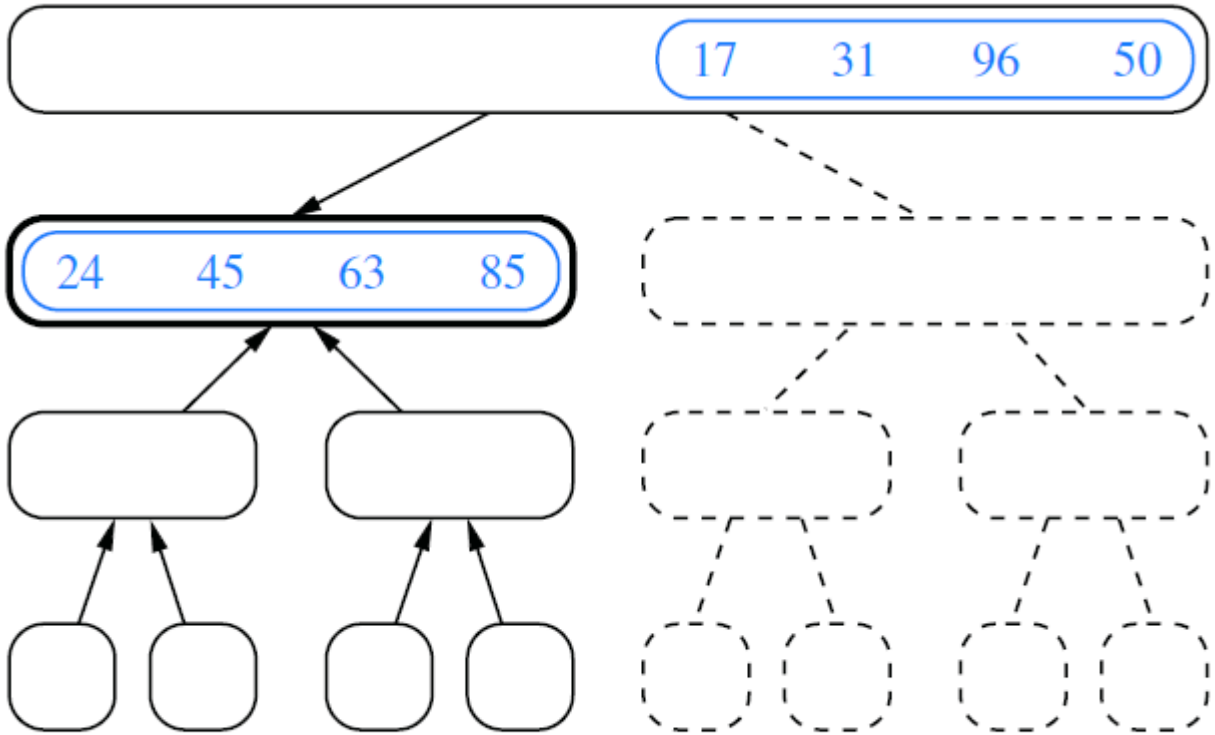


Merge-Sort

- **Merge-Sort Example**



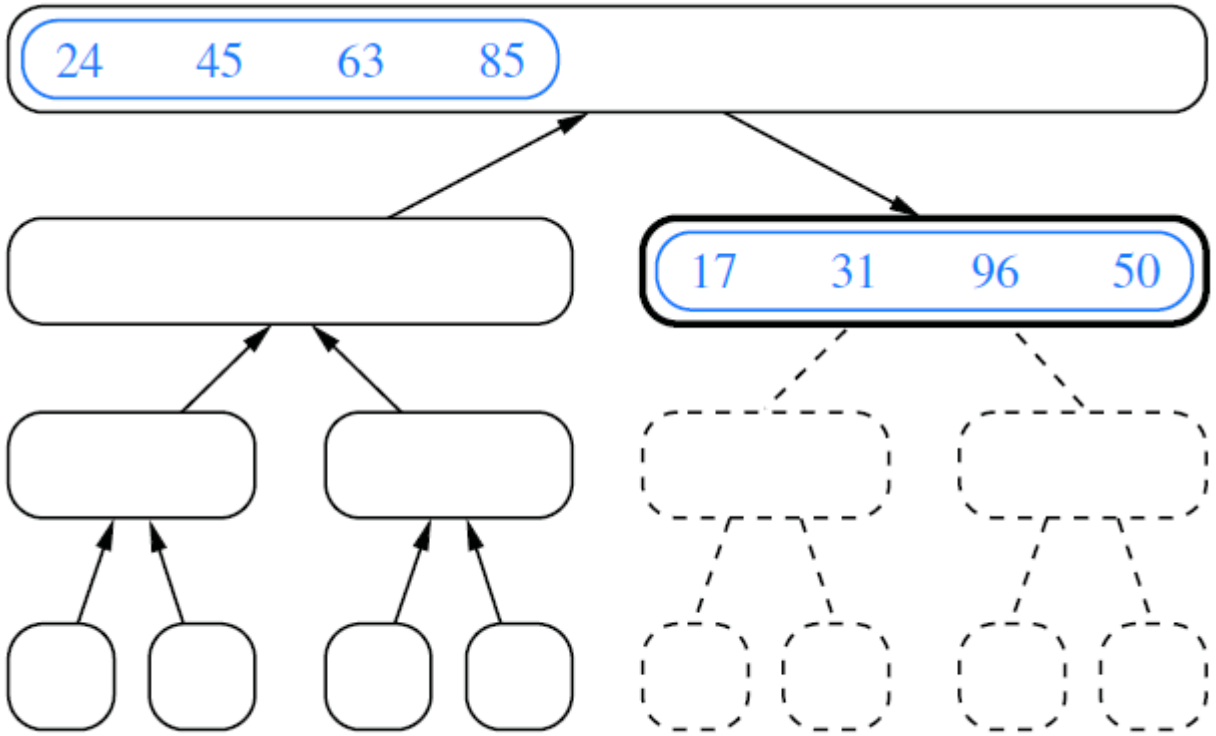
(k)



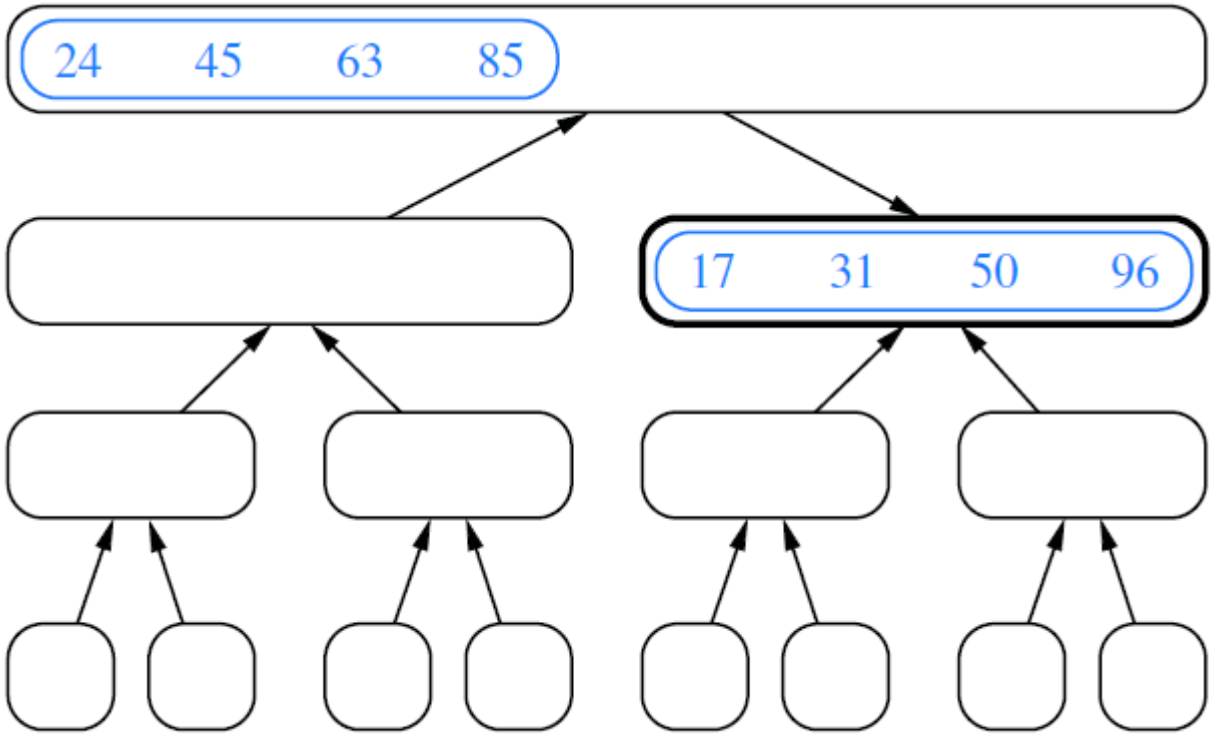
(l)

Merge-Sort

- **Merge-Sort Example**



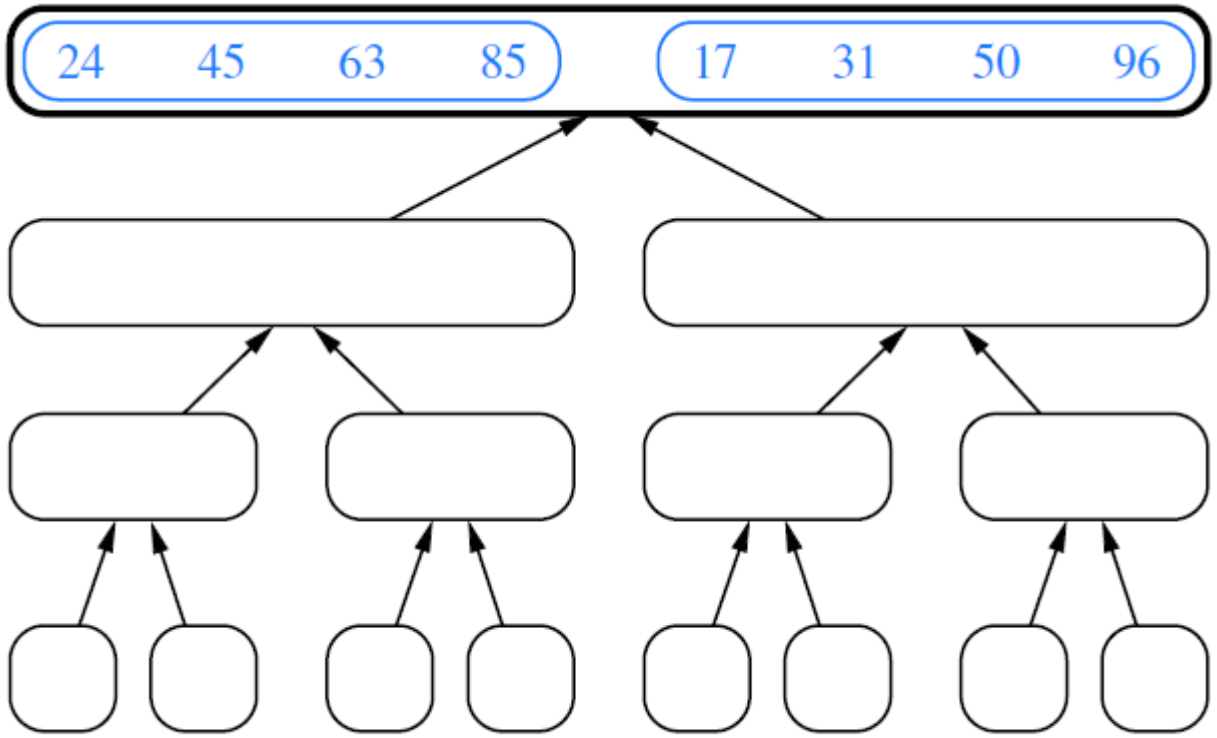
(m)



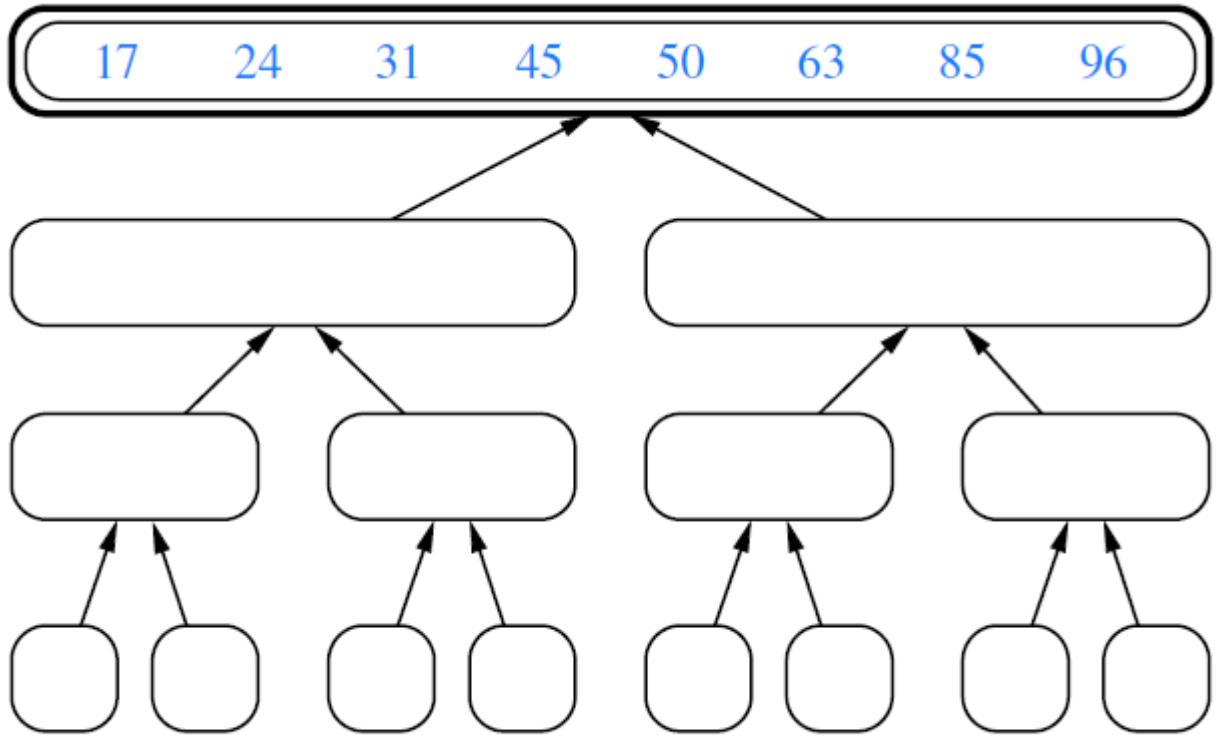
(n)

Merge-Sort

- **Merge-Sort Example**



(o)



(p)

▪ Merging Sorted Arrays

Algorithm merge(S_1, S_2, S):

Input: Sorted sequences S_1 and S_2 and an empty sequence S, all of which are implemented as arrays

Output: Sorted sequence S containing the elements from S_1 and S_2

```
i ← j ← 0
while i < S1.size() and j < S2.size() do
    if S1[i] ≤ S2[j] then
        S.insertBack(S1[i]) {copy ith element of S1 to end of S}
        i ← i + 1
    else
        S.insertBack(S2[j]) {copy jth element of S2 to end of S}
        j ← j + 1
while i < S1.size() do {copy the remaining elements of S1 to S}
    S.insertBack(S1[i])
    i ← i + 1
while j < S2.size() do {copy the remaining elements of S2 to S}
    S.insertBack(S2[j])
    j ← j + 1
```

Code Fragment 11.1: Algorithm for merging two sorted array-based sequences.

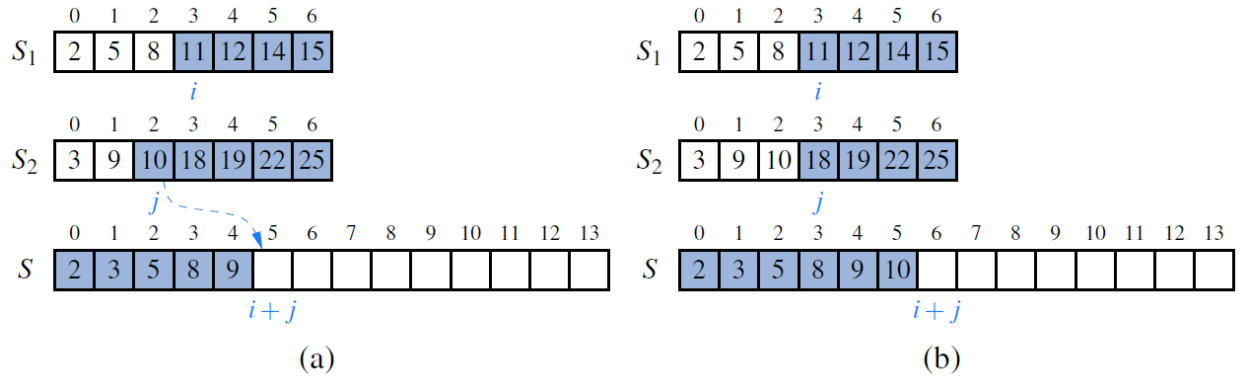


Figure 12.5: A step in the merge of two sorted arrays for which $S_2[j] < S_1[i]$. We show the arrays before the copy step in (a) and after it in (b).

Merge-Sort

- **Merging Sorted Lists**

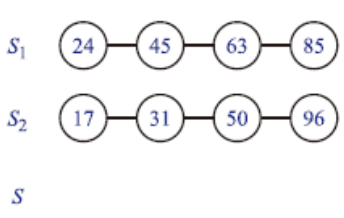
Algorithm merge(S_1, S_2, S):
Input: Sorted sequences S_1 and S_2 and an empty sequence S, implemented as linked lists
Output: Sorted sequence S containing the elements from S_1 and S_2

```
while S1 is not empty and S2 is not empty do
  if S1.front().element() ≤ S2.front().element() then
    {move the first element of S1 at the end of S}
    S.insertBack(S1.eraseFront())
  else
    {move the first element of S2 at the end of S}
    S.insertBack(S2.eraseFront())
{move the remaining elements of S1 to S}
while S1 is not empty do
  S.insertBack(S1.eraseFront())
{move the remaining elements of S2 to S}
while S2 is not empty do
  S.insertBack(S2.eraseFront())
```

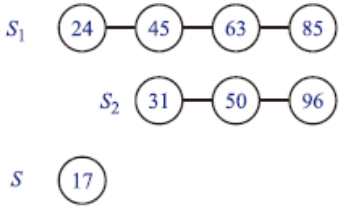
Code Fragment 11.2: Algorithm merge for merging two sorted sequences implemented as linked lists.

Merge-Sort

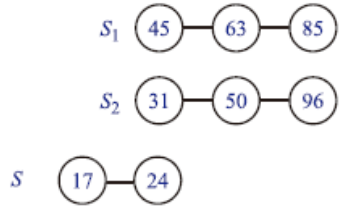
- **Merging Sorted Lists**



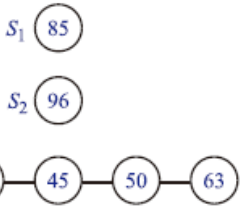
(a)



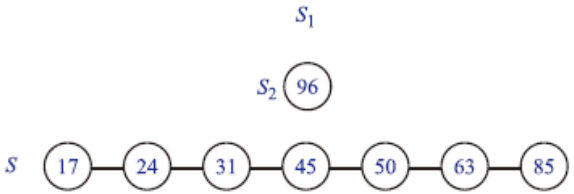
(b)



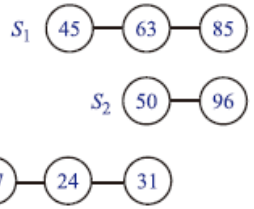
(c)



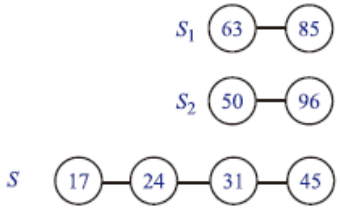
(g)



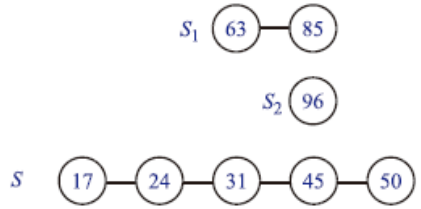
(h)



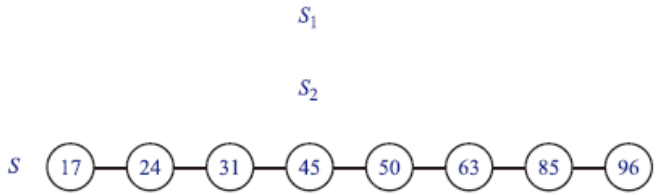
(d)



(e)



(f)



(i)

Merge-Sort

- **The Running Time for Merging**
  - Let n_1 and n_2 be the number of elements of S_1 and S_2, respectively
  - The operations inside each loop take $O(1)$ time each
  - One element is copied from either S_1 or S_2 to S
  - The overall number of iterations is $n_1 + n_2$, therefore, the running time of merge is $O(n_1 + n_2)$
- **The Running Time of Merge-Sort**
  - The time spent at a node v of T includes:
    - time to divide the sequence into two subsequences
    - time to combine the two sorted sequences

The Running Time of Merge-Sort

A merge-sort tree T, as portrayed in Figures 12.2 through 12.4, can guide our analysis. Consider a recursive call associated with a node v of the merge-sort tree T. The divide step at node v is straightforward; this step runs in time proportional to the size of the sequence for v, based on the use of slicing to create copies of the two list halves. We have already observed that the merging step also takes time that is linear in the size of the merged sequence. If we let i denote the depth of node v, the time spent at node v is $O(n/2^i)$, since the size of the sequence handled by the recursive call associated with v is equal to $n/2^i$.

Looking at the tree T more globally, as shown in Figure 12.6, we see that, given our definition of "time spent at a node," the running time of merge-sort is equal to the sum of the times spent at the nodes of T. Observe that T has exactly 2^i nodes at depth i. This simple observation has an important consequence, for it implies that the overall time spent at all the nodes of T at depth i is $O(2^i \cdot n/2^i)$, which is $O(n)$. By Proposition 12.1, the height of T is $\lceil \log n \rceil$. Thus, since the time spent at each of the $\lceil \log n \rceil + 1$ levels of T is $O(n)$, we have the following result:

Proposition 12.2: *Algorithm merge-sort sorts a sequence S of size n in $O(n \log n)$ time, assuming two elements of S can be compared in $O(1)$ time.*

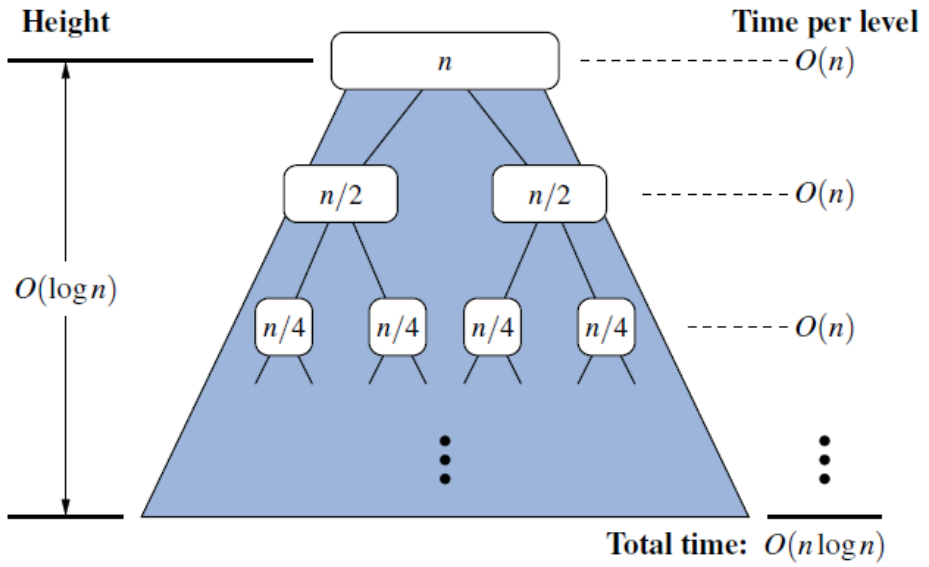


Figure 12.6: A visual analysis of the running time of merge-sort. Each node represents the time spent in a particular recursive call, labeled with the size of its subproblem.

Merge-Sort

- **C++ Implementations (List-based)**

```
template <typename E, typename C>                    // merge-sort S
void mergeSort(list<E>& S, const C& less) {
  typedef typename list<E>::iterator Itor;           // sequence of elements
  int n = S.size();
  if (n <= 1) return;                                // already sorted
  list<E> S1, S2;
  Itor p = S.begin();
  for (int i = 0; i < n/2; i++) S1.push_back(*p++); // copy first half to S1
  for (int i = n/2; i < n; i++) S2.push_back(*p++); // copy second half to S2
  S.clear();                                         // clear S's contents
  mergeSort(S1, less);                               // recur on first half
  mergeSort(S2, less);                               // recur on second half
  merge(S1, S2, S, less);                            // merge S1 and S2 into S
}
```

85	24	63	45	17	31	96	50

17	24	31	45	50	63	85	96

```
template <typename E, typename C>                    // merge utility
void merge(list<E>& S1, list<E>& S2, list<E>& S, const C& less) {
  typedef typename list<E>::iterator Itor;           // sequence of elements
  Itor p1 = S1.begin();
  Itor p2 = S2.begin();
  while(p1 != S1.end() && p2 != S2.end()) {          // until either is empty
    if(less(*p1, *p2))                               // append smaller to S
      S.push_back(*p1++);
    else
      S.push_back(*p2++);
  }
  while(p1 != S1.end())                              // copy rest of S1 to S
    S.push_back(*p1++);
  while(p2 != S2.end())                              // copy rest of S2 to S
    S.push_back(*p2++);
}
```

Code Fragment 11.3: Functions mergeSort and merge implementing a list-based merge-sort algorithm.

Merge-Sort

- **C++ Implementations (Vector-based)**

```
template <typename E, typename C>                    // merge-sort S
void mergeSort(vector<E>& S, const C& less) {
  typedef vector<E> vect;
  int n = S.size();
  vect v1(S);  vect* in = &v1;                       // initial input vector
  vect v2(n);  vect* out = &v2;                      // initial output vector
  for (int m = 1; m < n; m *= 2) {                   // list sizes doubling
    for (int b = 0; b < n; b += 2*m) {               // beginning of list
      merge(*in, *out, less, b, m);                  // merge sublists
    }
    std::swap(in, out);                              // swap input with output
  }
  S = *in;                                           // copy sorted array to S
}
```

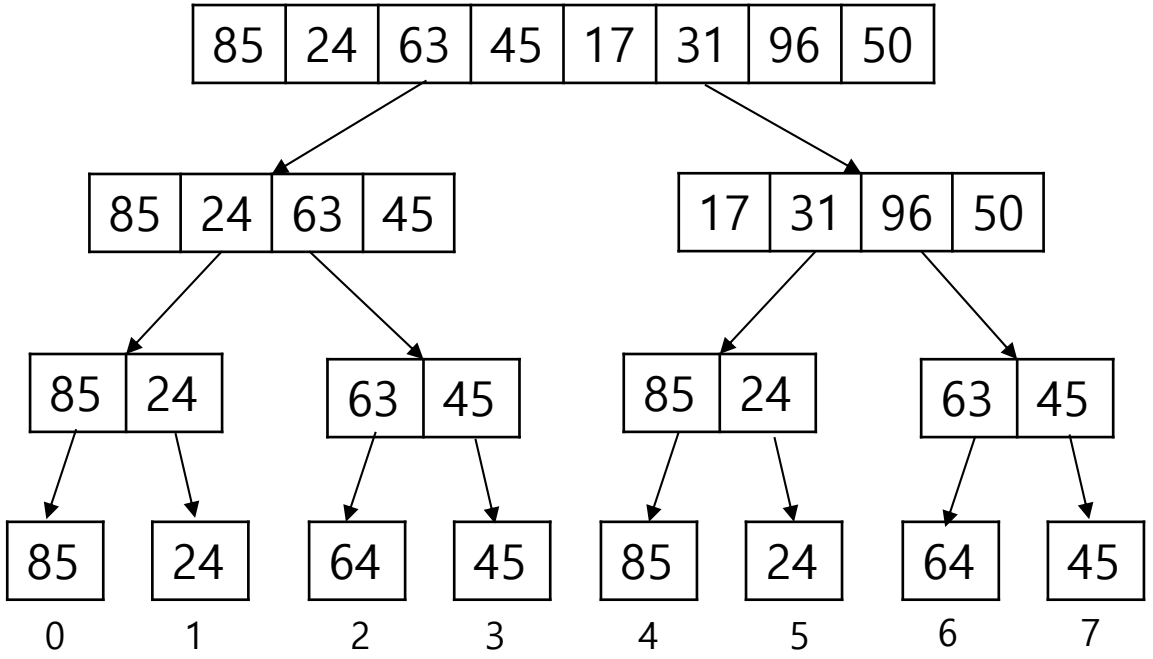
```
                    // merge in[b..b+m-1] and in[b+m..b+2*m-1]
template <typename E, typename C>
void merge(vector<E>& in, vector<E>& out, const C& less, int b, int m) {
  int i = b;                                         // index into run #1
  int j = b + m;                                     // index into run #2
  int n = in.size();
  int e1 = std::min(b + m, n);                       // end of run #1
  int e2 = std::min(b + 2*m, n);                     // end of run #2
  int k = b;
  while ((i < e1) && (j < e2)) {
    if(!less(in[j], in[i]))                          // append smaller to S
      out[k++] = in[i++];
    else
      out[k++] = in[j++];
  }
  while (i < e1)                                     // copy rest of run 1 to S
    out[k++] = in[i++];
  while (j < e2)                                     // copy rest of run 2 to S
    out[k++] = in[j++];
}
```

85	24	63	45	17	31	96	50

17	24	31	45	50	63	85	96

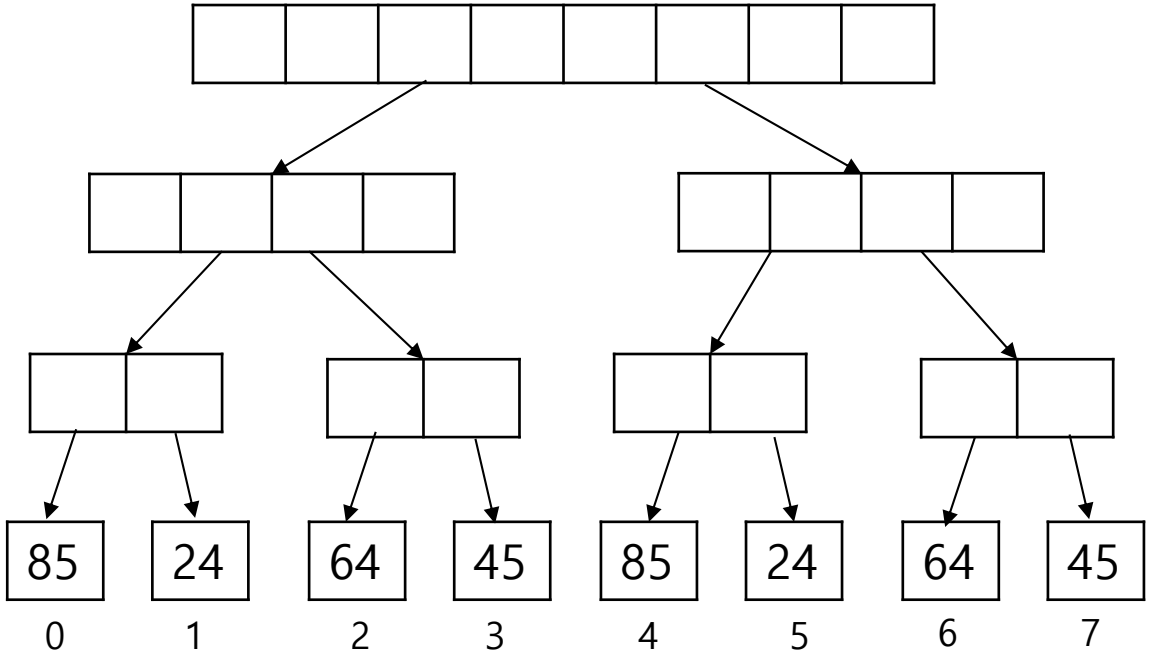
Code Fragment 11.4: Functions mergeSort and merge implementing a vector-based merge-sort algorithm. We employ the STL functions swap, which swaps two values, and min, which returns the minimum of two values. Note that the call to swap swaps only the pointers to the arrays and, hence, runs in $O(1)$ time.

Merge-Sort



- m=1
  - b=0
    - Merge [0,0], [1,1]
  - b=2
    - Merge [2,2] and [3,3]
  - b=4
    - Merge [4,4] and [5,5]
  - b=6
    - Merge [6,6] and [7,7]
- m=2
  - b=0
    - Merge [0,1] and [2,3]
  - b=4
    - Merge [4,5] and [6,7]
- m=4
  - b=0
    - Merge[1,3] and [4,7]

Merge-Sort



- m=1
  - b=0
    - Merge [0,0], [1,1]
  - b=2
    - Merge [2,2] and [3,3]
  - b=4
    - Merge [4,4] and [5,5]
  - b=6
    - Merge [6,6] and [7,7]
- m=2
  - b=0
    - Merge [0,1] and [2,3]
  - b=4
    - Merge [4,5] and [6,7]
- m=4
  - b=0
    - Merge[1,3] and [4,7]

Quick-Sort

- **Quick-Sort**
  - A divide-and-conquer sorting algorithm
  - Partitions elements into two subsequences using a pivot

1. ***Divide:*** If S has at least two elements (nothing needs to be done if S has zero or one element), select a specific element x from S, which is called the ***pivot***. As is common practice, choose the pivot x to be the last element in S. Remove all the elements from S and put them into three sequences:
   - L, storing the elements in S less than x
   - E, storing the elements in S equal to x
   - G, storing the elements in S greater than x

   Of course, if the elements of S are distinct, then E holds just one element—the pivot itself.
2. ***Conquer:*** Recursively sort sequences L and G.
3. ***Combine:*** Put back the elements into S in order by first inserting the elements of L, then those of E, and finally those of G.

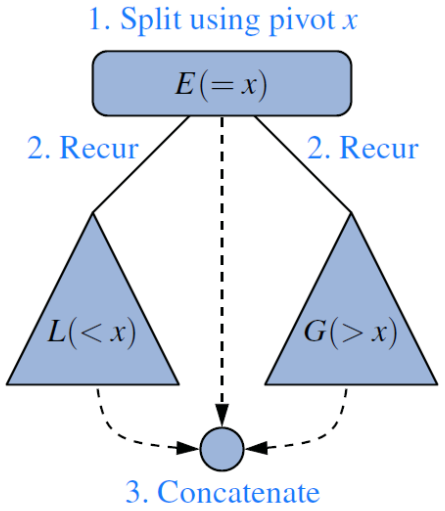


Figure 12.8: A visual schematic of the quick-sort algorithm.

Quick-Sort

- **Quick-Sort Tree**
  - The height of the quick-sort tree is linear in the worst case (*e.g.*, already sorted sequence)

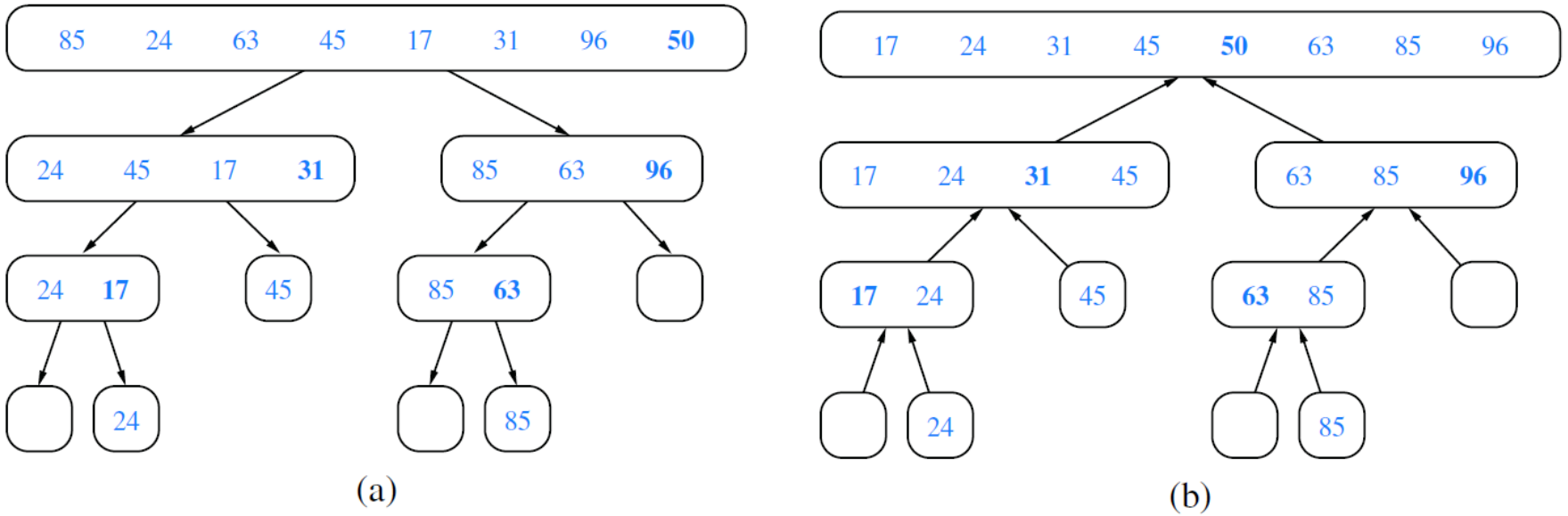
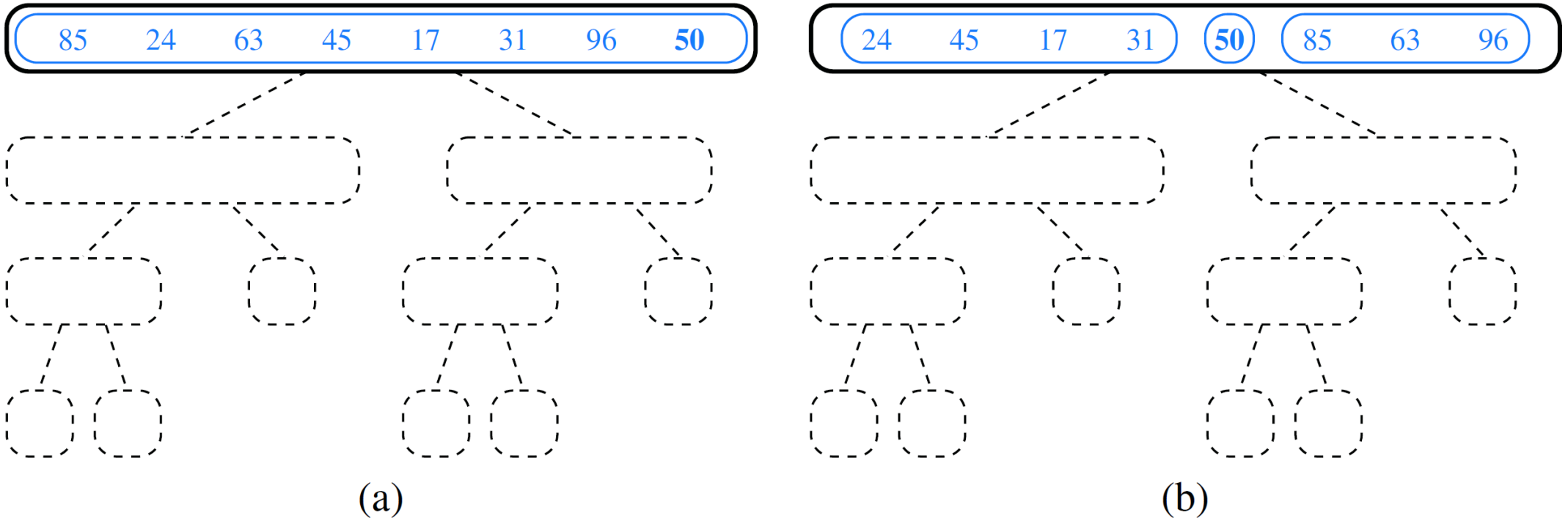


Figure 12.9: Quick-sort tree T for an execution of the quick-sort algorithm on a sequence with 8 elements: (a) input sequences processed at each node of T; (b) output sequences generated at each node of T. The pivot used at each level of the recursion is shown in bold.

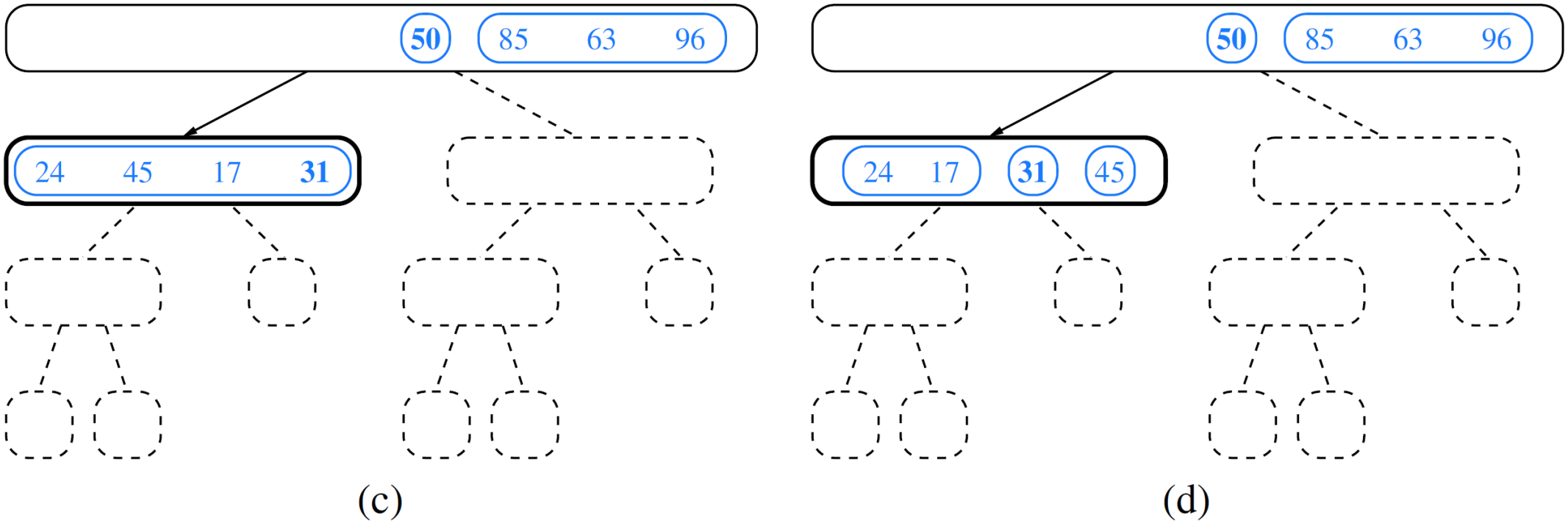
Quick-Sort

- **Quick-Sort Example**



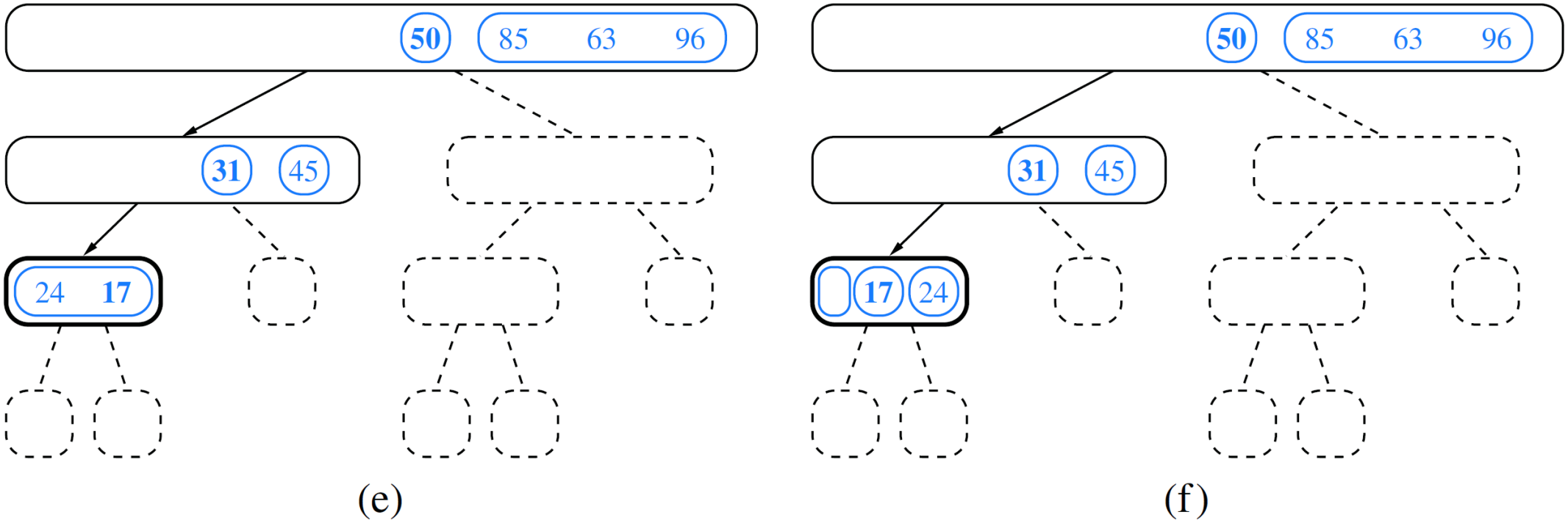
Quick-Sort

- **Quick-Sort Example**



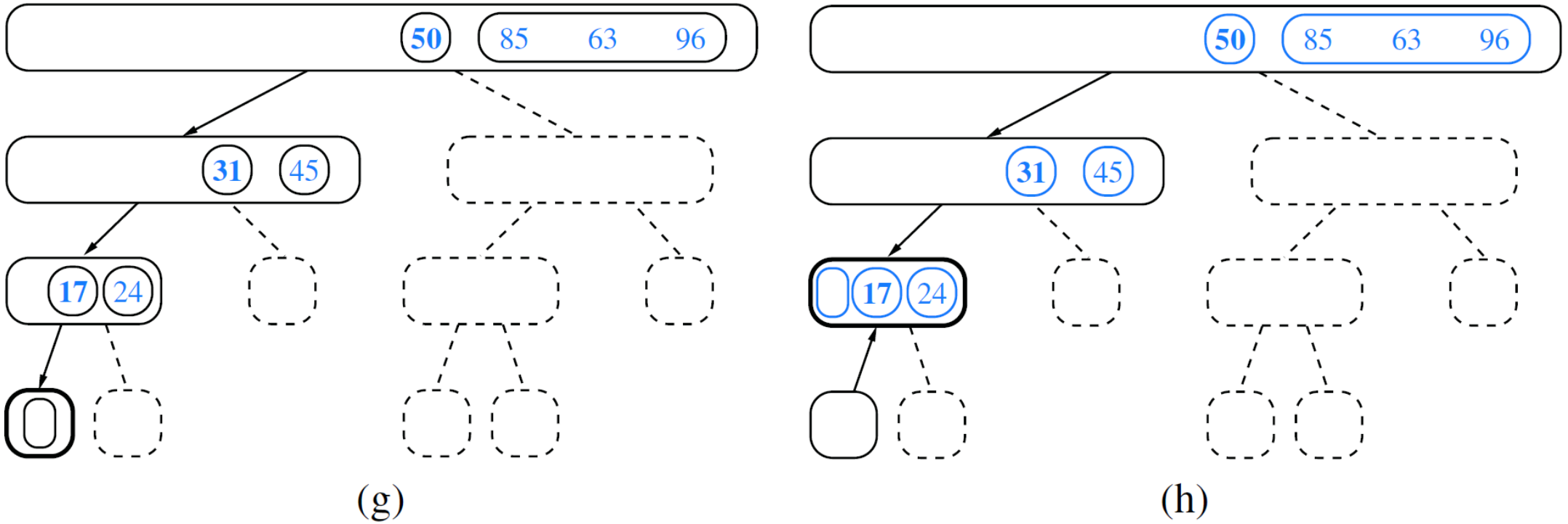
Quick-Sort

- **Quick-Sort Example**



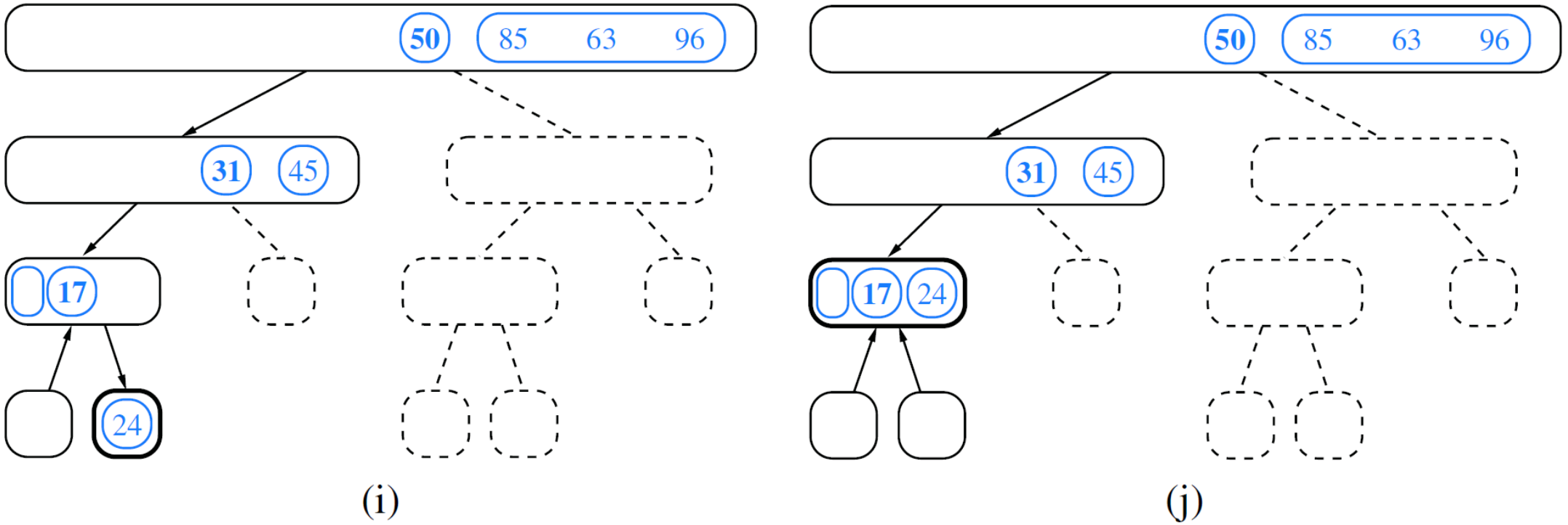
Quick-Sort

- **Quick-Sort Example**

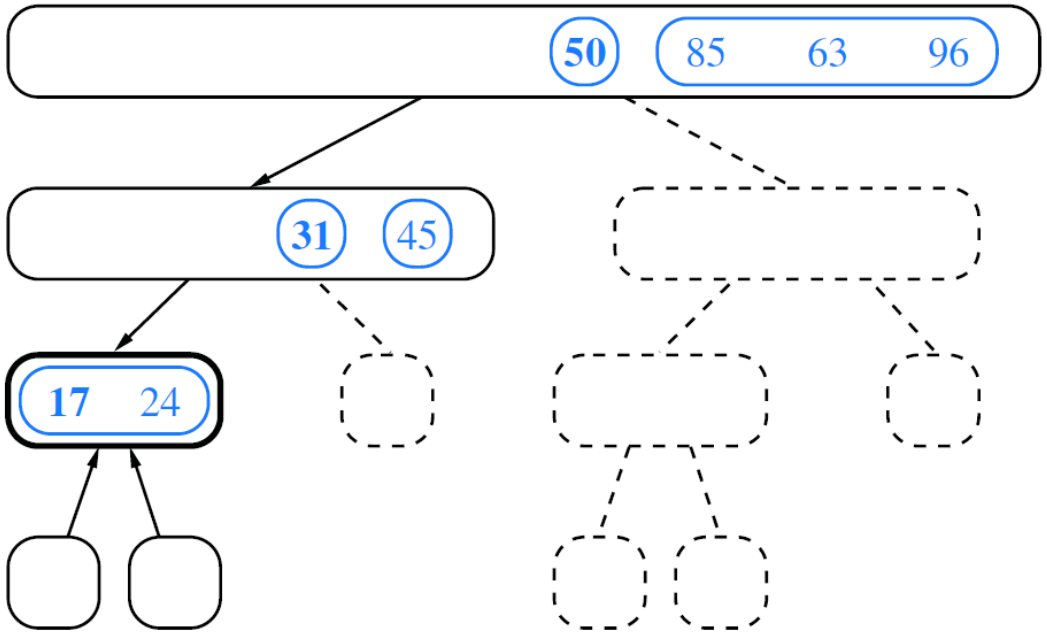


Quick-Sort

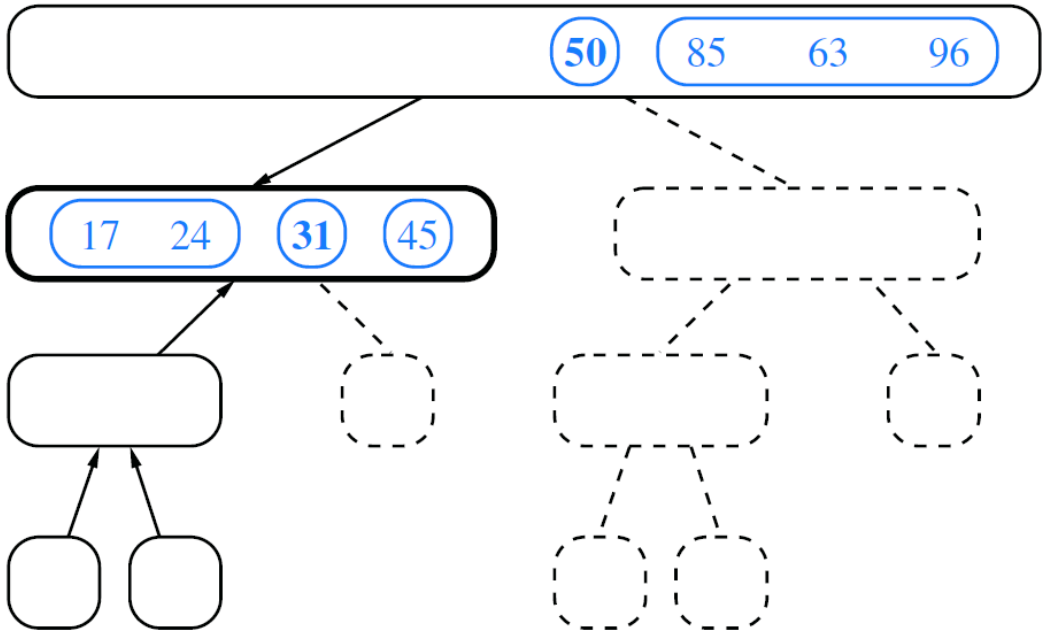
- **Quick-Sort Example**



- **Quick-Sort Example**

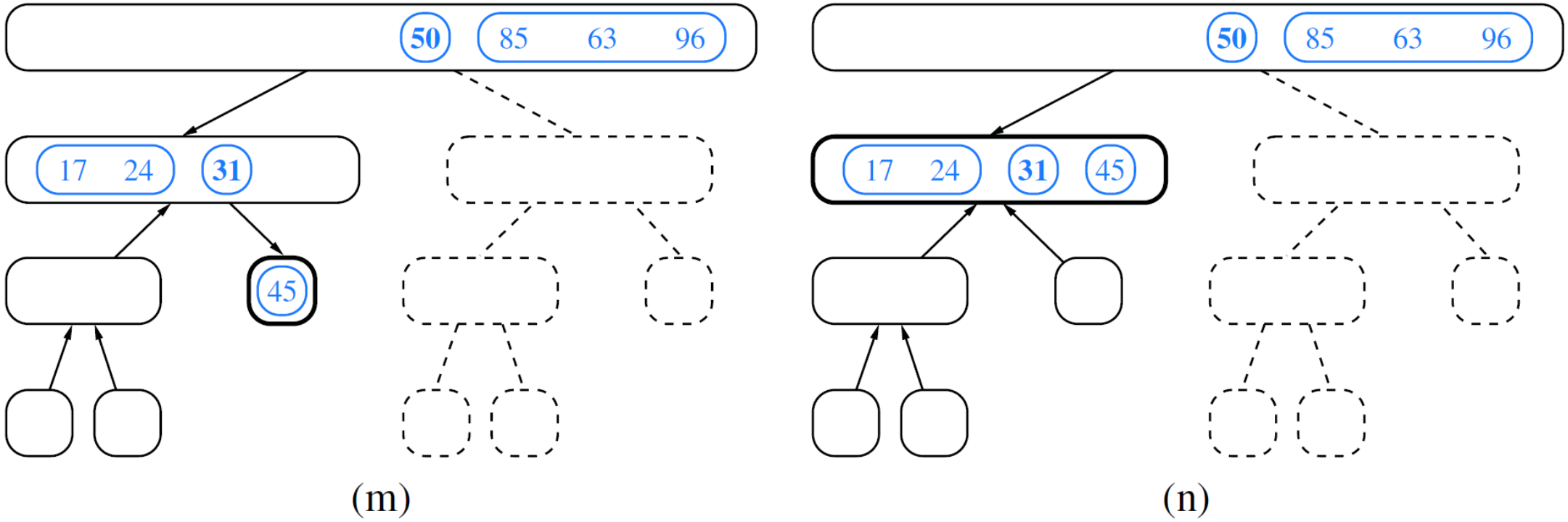


(k)



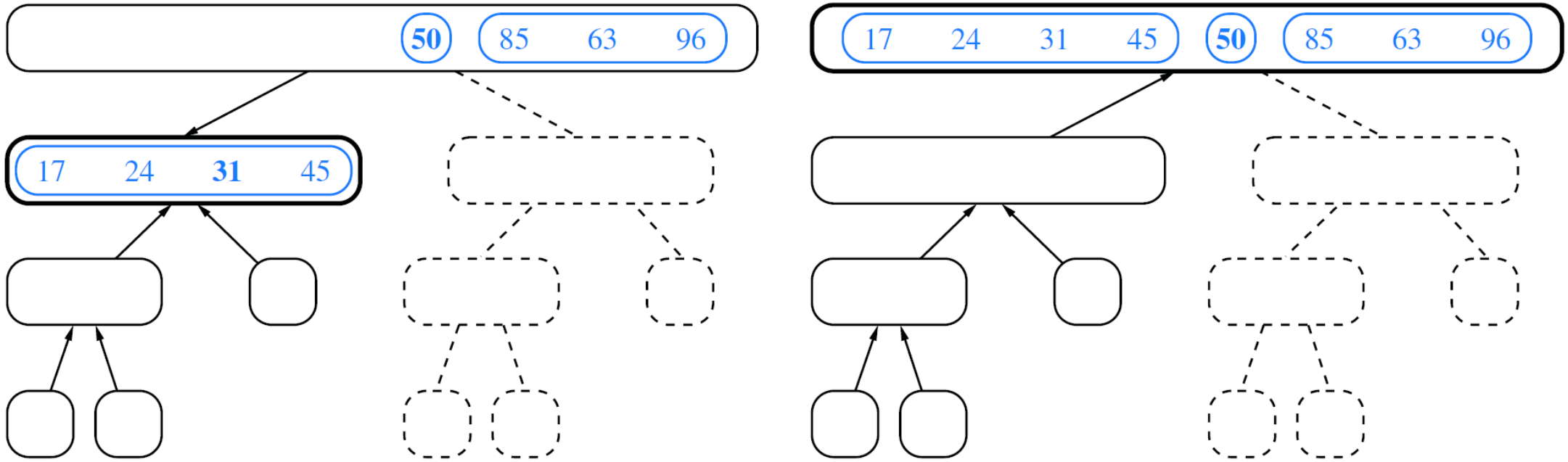
(l)

- **Quick-Sort Example**



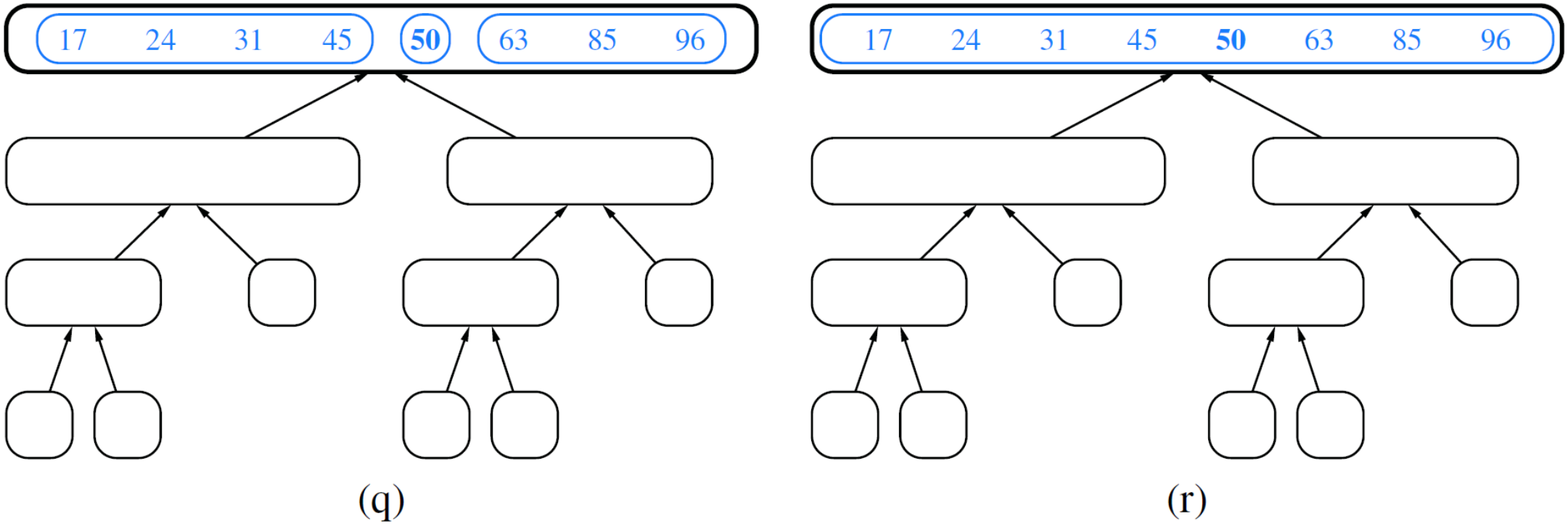
Quick-Sort

- **Quick-Sort Example**



Quick-Sort

- **Quick-Sort Example**



Quick-Sort

- **C++ Implementation**

```
Algorithm QuickSort(S):
    Input: A sequence S implemented as an array or linked list
    Output: The sequence S in sorted order
    if S.size() ≤ 1 then
        return                {S is already sorted in this case}
    p ← S.back().element()    {the pivot}
    Let L, E, and G be empty list-based sequences
    while !S.empty() do {scan S backwards, dividing it into L, E, and G}
        if S.back().element() < p then
            L.insertBack(S.eraseBack())
        else if S.back().element() = p then
            E.insertBack(S.eraseBack())
        else {the last element in S is greater than p}
            G.insertBack(S.eraseBack())
    QuickSort(L)              {Recur on the elements less than p}
    QuickSort(G)              {Recur on the elements greater than p}
    while !L.empty() do {copy back to S the sorted elements less than p}
        S.insertBack(L.eraseFront())
    while !E.empty() do {copy back to S the elements equal to p}
        S.insertBack(E.eraseFront())
    while !G.empty() do {copy back to S the sorted elements greater than p}
        S.insertBack(G.eraseFront())
    return                    {S is now in sorted order}
```

Code Fragment 11.5: Quick-sort for an input sequence S implemented with a linked list or an array.

Quick-Sort

- **The Running Time of Quick-Sort**
  - Input size $s(v)$
    - The size of the sequence handled by the invocation of quick-sort associated with node v
  - The sum of input sizes of the children of v is at most $s(v) - 1$ (due to the pivot)

Let s_i denote the sum of the input sizes of the nodes at depth i for a particular quick-sort tree T. Clearly, $s_0 = n$, since the root r of T is associated with the entire sequence. Also, $s_1 \leq n - 1$, since the pivot is not propagated to the children of r. More generally, it must be that $s_i < s_{i-1}$ since the elements of the subsequences at depth i all come from distinct subsequences at depth $i - 1$, and at least one element from depth $i - 1$ does not propagate to depth i because it is in a set E (in fact, one element from *each node* at depth $i - 1$ does not propagate to depth i).

We can therefore bound the overall running time of an execution of quick-sort as $O(n \cdot h)$ where h is the overall height of the quick-sort tree T for that execution. Unfortunately, in the worst case, the height of a quick-sort tree is $n - 1$, as observed in Section 12.2. Thus, quick-sort runs in $O(n^2)$ worst-case time. Paradoxically, if we choose the pivot as the last element of the sequence, this worst-case behavior occurs for problem instances when sorting should be easy—if the sequence is already sorted.

Quick-Sort

- **The Running Time of Quick-Sort**
  - Input size $s(v)$
    - The size of the sequence handled by the invocation of quick-sort associated with node v
  - The sum of input sizes of the children of v is at most $s(v) - 1$ (due to the pivot)

Going back to our analysis, note that the best case for quick-sort on a sequence of distinct elements occurs when subsequences L and G happen to have roughly the same size. That is, in the best case, we have

$$\begin{aligned} s_0 &= n \\ s_1 &= n-1 \\ s_2 &= n-(1+2) = n-3 \\ &\vdots \\ s_i &= n-(1+2+2^2+\cdots+2^{i-1}) = n-(2^i-1). \end{aligned}$$

Thus, in the best case, T has height $O(\log n)$ and quick-sort runs in $O(n \log n)$ time.

Randomized Quick-Sort

- **Picking Pivots at Random**
  - Selecting a pivot close to the middle of the set leads to an $O(n \log n)$ running time
  - Picks a random element of the input sequence as the pivot

Proposition 12.3: *The expected running time of randomized quick-sort on a sequence S of size n is $O(n \log n)$.*

Justification: Let S be a sequence with n elements and let T be the binary tree associated with an execution of randomized quick-sort on S. First, we observe that the running time of the algorithm is proportional to the number of comparisons performed. We consider the recursive call associated with a node of T and observe that during the call, all comparisons are between the pivot element and another element of the input of the call. Thus, we can evaluate the total number of comparisons performed by the algorithm as $\sum_{s \in S} C(x)$, where $C(x)$ is the number of comparisons involving x as a nonpivot element. Next, we will show that for every element $x \in S$, the expected value of $C(x)$ is $O(\log n)$. Since the expected value of a sum is the sum of the expected values of its terms, an $O(\log n)$ bound on the expected value of $C(x)$ implies that randomized quick-sort runs in expected $O(n \log n)$ time.

Randomized Quick-Sort

▪ Picking Pivots at Random

To show that the expected value of $C(x)$ is $O(n \log n)$ for any x, we fix an arbitrary element x and consider the path of nodes in the tree T associated with recursive calls for which x is part of the input sequence. (See Figure 12.13.) By definition, $C(x)$ is equal to that path length, as x will take part in one nonpivot comparison per level of the tree until it is chosen as the pivot or is the only element that remains.

Let n_d denote the input size for the node of that path at depth d of tree T, for $0 \leq d \leq C(x)$. Since all elements are in the initial recursive call, $n_0 = n$. We know that the input size for any recursive call is at least one less than the size of its parent, and thus that $n_{d+1} \leq n_d - 1$ for any $d < C(x)$. In the worst case, this implies that $C(x) \leq n - 1$, as the recursive process stops if $n_d = 1$ or if x is chosen as the pivot.

We can show the stronger claim that the expected value of $C(x)$ is $O(\log n)$ based on the random selection of a pivot at each level. The choice of pivot at depth d of this path is considered "good" if $n_{d+1} \leq 3n_d/4$. The choice of a pivot will be good with probability at least $1/2$, as there are at least $n_d/2$ elements in the input that, if chosen as pivot, will result in at least $n_d/4$ elements begin placed in each subproblem, thereby leaving x in a group with at most $3n_d/4$ elements.

We conclude by noting that there can be at most $\log_{4/3} n$ such good pivot choices before x is isolated. Since a choice is good with probability at least 1/2, the expected number of recursive calls before achieving $\log_{4/3} n$ good choices is at most $2 \log_{4/3} n$, which implies that $C(x)$ is $O(\log n)$.

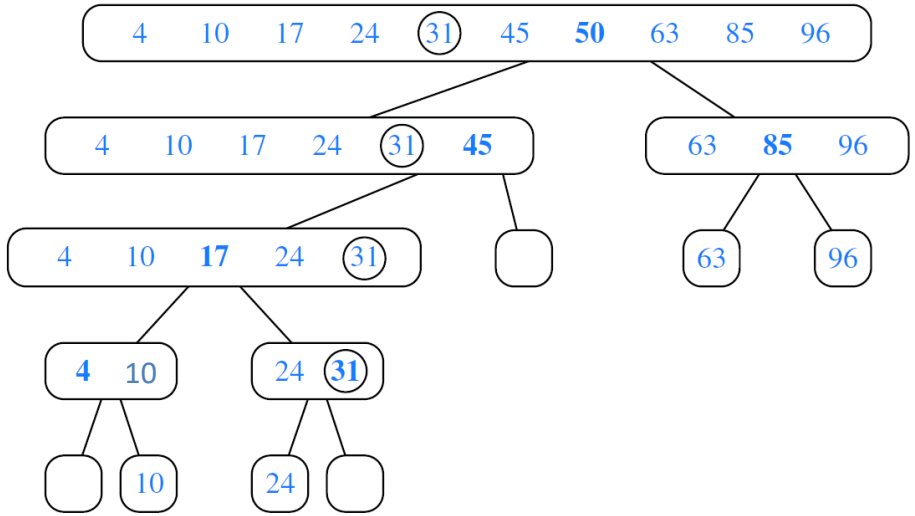


Figure 12.13: An illustration of the analysis of Proposition 12.3 for an execution of randomized quick-sort. We focus on element $x = 31$, which has value $C(x) = 3$, as it is the nonpivot element in a comparison with 50, 45, and 17. By our notation, $n_0 = 10$, $n_1 = 6$, $n_2 = 5$, and $n_3 = 2$, and the pivot choices of 50 and 17 are good.

$$n_i = \left(\frac{3}{4}\right)^i n$$

Randomized Quick-Sort

- **Picking Pivots at Random**

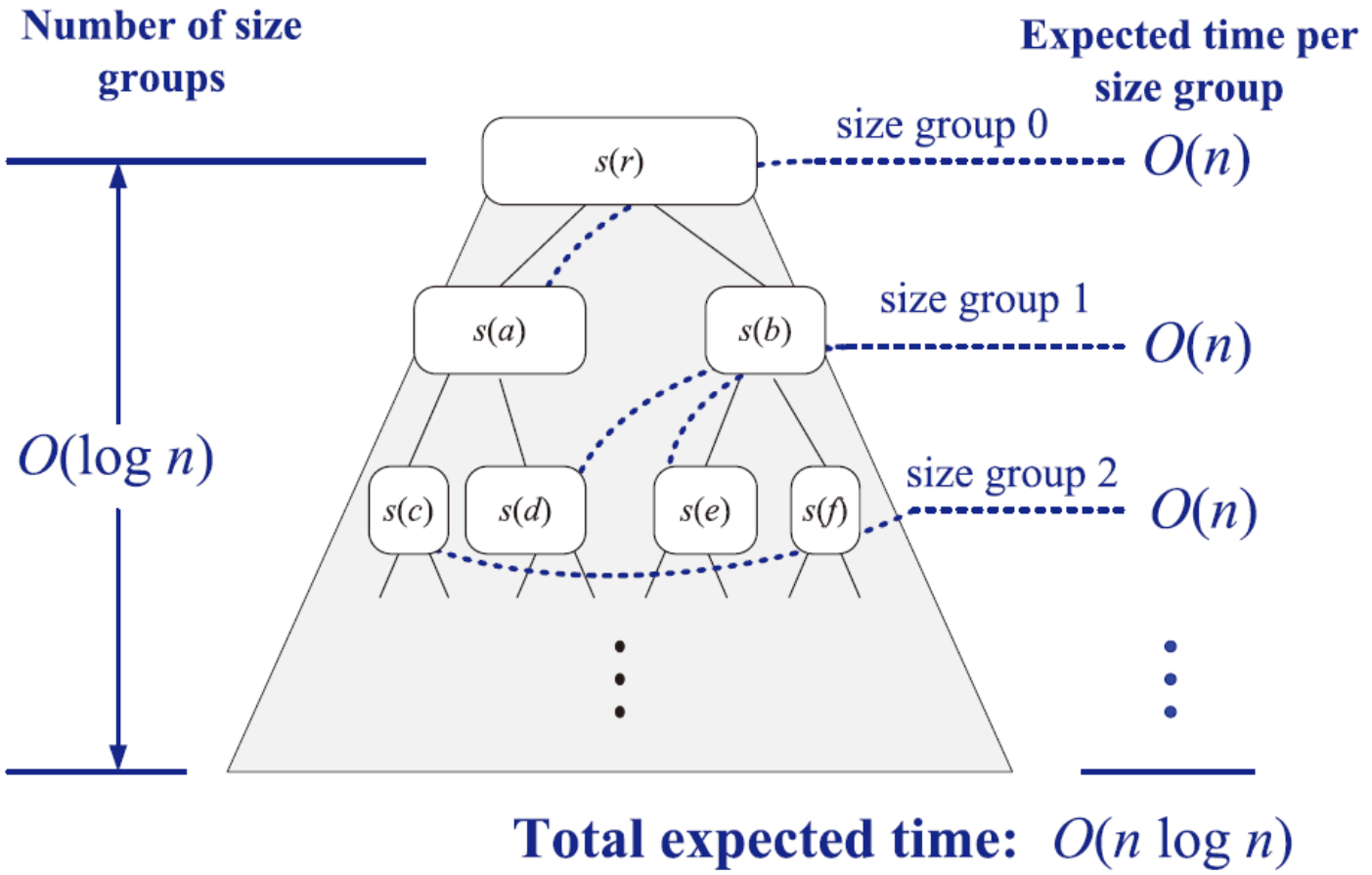


Figure 11.13: A visual time analysis of the quick-sort tree T. Each node is shown labeled with the size of its subproblem.

In-Place Quick-Sort

- **C++ Implementation**

Algorithm inPlaceQuickSort(S,a,b):
Input: An array S of distinct elements; integers a and b
Output: Array S with elements originally from indices from a to b, inclusive, sorted in nondecreasing order from indices a to b

```
if a ≥ b then return          {at most one element in subrange}
p ← S[b]                      {the pivot}
l ← a                         {will scan rightward}
r ← b − 1                     {will scan leftward}
while l ≤ r do
    {find an element larger than the pivot}
    while l ≤ r and S[l] ≤ p do
        l ← l + 1
    {find an element smaller than the pivot}
    while r ≥ l and S[r] ≥ p do
        r ← r − 1
    if l < r then
        swap the elements at S[l] and S[r]
{put the pivot into its final place}
swap the elements at S[l] and S[b]
{recursive calls}
inPlaceQuickSort(S, a, l − 1)
inPlaceQuickSort(S, l + 1, b)
{we are done at this point, since the sorted subarrays are already consecutive}
```

Code Fragment 11.6: In-place quick-sort for an input array S.

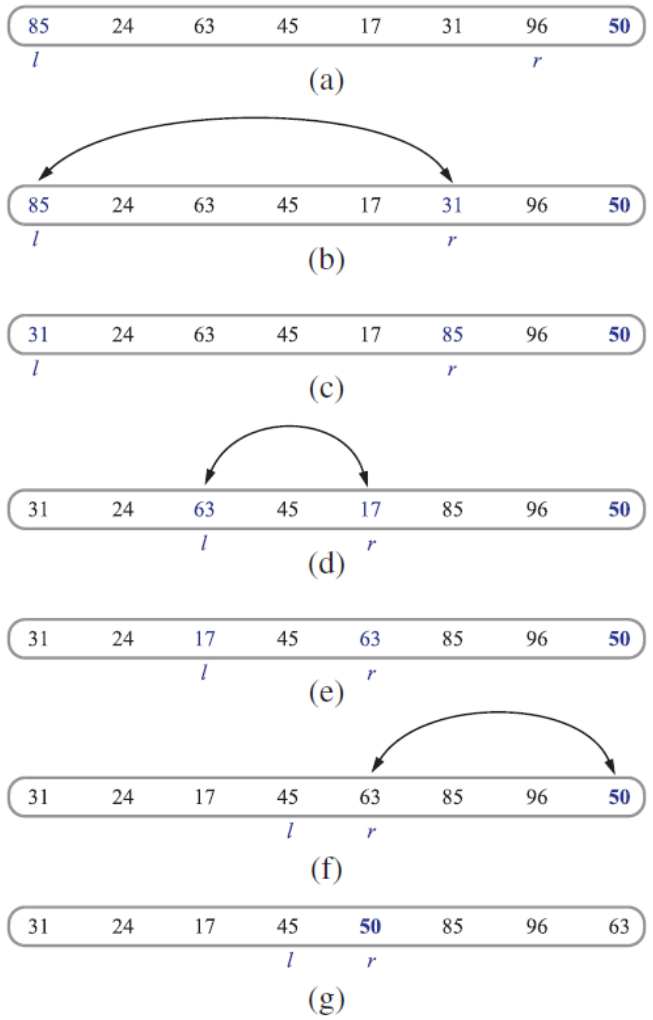


Figure 11.14: Divide step of in-place quick-sort. Index l scans the sequence from left to right, and index r scans the sequence from right to left. A swap is performed when l is at an element larger than the pivot and r is at an element smaller than the pivot. A final swap with the pivot completes the divide step.

In-Place Quick-Sort

- **C++ Implementation**

```
template <typename E, typename C>                    // quick-sort S
void quickSort(std::vector<E>& S, const C& less) {
  if (S.size() <= 1) return;                         // already sorted
  quickSortStep(S, 0, S.size()-1, less);             // call sort utility
}
template <typename E, typename C>
void quickSortStep(std::vector<E>& S, int a, int b, const C& less) {
  if (a >= b) return;                                // 0 or 1 left? done
  E pivot = S[b];                                    // select last as pivot
  int l = a;                                         // left edge
  int r = b - 1;                                     // right edge
  while (l <= r) {
    while (l <= r && !less(pivot, S[l])) l++;        // scan right till larger
    while (r >= l && !less(S[r], pivot)) r--;        // scan left till smaller
    if (l < r)                                       // both elements found
      std::swap(S[l], S[r]);
  }                                                  // until indices cross
  std::swap(S[l], S[b]);                             // store pivot at l
  quickSortStep(S, a, l-1, less);                    // recur on both sides
  quickSortStep(S, l+1, b, less);
}
```

Code Fragment 11.7: A coding of in-place quick-sort, assuming distinct elements.

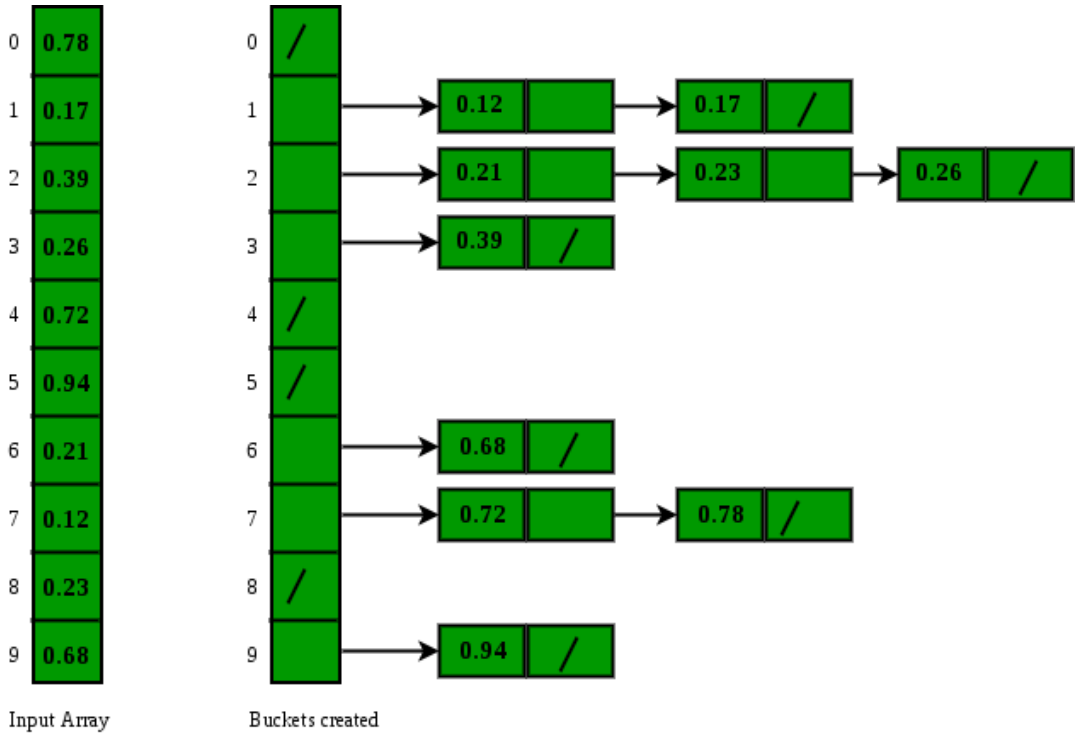
Linear-Time Sorting

Linear-Time Sorting

- **Bucket-Sort**
  - Consider a sequence S of n entries
  - Keys are integers in the range $[0, N-1]$ for $N \geq 2$
  - Bucket-sort takes $O(n+N)$ time
  - Uses keys as indices into a bucket array B
  - Efficient when N is $O(n)$ or $O(n \log n)$

Algorithm bucketSort(S):
Input: Sequence S of entries with integer keys in the range $[0, \mathrm{N}-1]$
Output: Sequence S sorted in nondecreasing order of the keys
let B be an array of n sequences, each of which is initially empty
for each entry e in S **do**
  let k denote the key of e
  remove e from S and insert it at the end of bucket (sequence) $B[k]$
for $i = 0$ to $n-1$ **do**
  for each entry e in sequence $B[i]$ **do**
    remove e from $B[i]$ and insert it at the end of S

https://www.geeksforgeeks.org/bucket-sort-2



Linear-Time Sorting

- **Stable Sorting**
  - Let $S = \big((k_0, x_0), \ldots, (k_{n-1}, x_{n-1})\big)$
  - A sorting algorithm is stable if relative ordering of any two entries with the same key is the same before and after sorting

When sorting key-value pairs, an important issue is how equal keys are handled. Let $S = ((k_0, v_0), \ldots, (k_{n-1}, v_{n-1}))$ be a sequence of such entries. We say that a sorting algorithm is ***stable*** if, for any two entries (k_i, v_i) and (k_j, v_j) of S such that $k_i = k_j$ and (k_i, v_i) precedes (k_j, v_j) in S before sorting (that is, $i < j$), entry (k_i, v_i) also precedes entry (k_j, v_j) after sorting. Stability is important for a sorting algorithm because applications may want to preserve the initial order of elements with the same key.

Linear-Time Sorting

- **Radix-Sort**
  - Entries have keys that are pairs (k, l) where k and l are integers in the range $[0, N-1]$
  - $(k_1, l_1) < (k_2, l_2)$:
    - if $k_1 < k_2$
    - if $k_1 = k_2$ and $l_1 < l_2$
  - The radix-sort algorithm sorts a sequence S of entries with pair keys
  - Applying a stable bucket-sort on the sequence twice
    - First sort using the second component, and then using the first component

Proposition 12.6: *Let S be a sequence of n key-value pairs, each of which has a key $(k_1, k_2, \ldots, k_d)$, where k_i is an integer in the range $[0, N-1]$ for some integer $N \geq 2$. We can sort S lexicographically in time $O(d(n+N))$ using radix-sort.*

Linear-Time Sorting

- **Radix-Sort Example**

Example 12.5: *Consider the following sequence S (we show only the keys):*

$$S = ((3,3),(1,5),(2,5),(1,2),(2,3),(1,7),(3,2),(2,2)).$$

If we sort S stably on the first component, then we get the sequence

$$S_1 = ((1,5),(1,2),(1,7),(2,5),(2,3),(2,2),(3,3),(3,2)).$$

If we then stably sort this sequence S_1 using the second component, we get the sequence

$$S_{1,2} = ((1,2),(2,2),(3,2),(2,3),(3,3),(1,5),(2,5),(1,7)),$$

which is unfortunately not a sorted sequence. On the other hand, if we first stably sort S using the second component, then we get the sequence

$$S_2 = ((1,2),(3,2),(2,2),(3,3),(2,3),(1,5),(2,5),(1,7)).$$

If we then stably sort sequence S_2 using the first component, we get the sequence

$$S_{2,1} = ((1,2),(1,5),(1,7),(2,2),(2,3),(2,5),(3,2),(3,3)),$$

which is indeed sequence S lexicographically ordered.

Comparing Sorting Algorithms

- **Insertion-Sort**
  - $O(n+m)$ where m is the number of inversions
  - Simple to program
  - Effective for sorting almost sorted sequences
- **Merge-sort**
  - $O(n \log n)$ in the worst-case
  - Difficult to make merge-sort run in-place
- **Quick-Sort**
  - $O(n^2)$ in the worst-case, $O(n \log n)$ expected running time
  - An excellent choice as a general-purpose in-place sorting algorithm
- **Heap-Sort**
  - $O(n \log n)$ in the worst-case
  - Can be made to execute in-place
- **Bucket-Sort and Radix-Sort**
  - $O\big(d(n+N)\big)$ where $[0, N-1]$ is the range of integer keys ($d = 1$ for bucket sort)
  - Keys are small integers